



OPEN Model-based partition scheduling of integrated modular avionics systems using genetic algorithm

Jichen Chen¹, Zhengjun Zhai¹✉, Pujie Han² & Min Huang²✉

This paper proposes a model-based method for partition optimization in Integrated Modular Avionics (IMA) systems that comply with the ARINC 653 standard. It is a complex combinatorial optimization problem to schedule a variety of real-time partitions to achieve maximum processor utilization in an IMA system. Our approach effectively utilizes timed automata in UPPAAL to address the modeling challenges posed by the ARINC 653 features, including two-level scheduling, multi-core processors, and both strictly and loosely periodic partitions. A parallel genetic algorithm is employed to explore the scheduling parameter space of the IMA model, accelerating the computationally expensive model-based analysis through parallelization and achieving an optimal solution more efficiently than traditional analytical methods. Furthermore, we propose a compositional framework that allows for the independent verification of each partition while aggregating local results to derive global schedulability properties, thereby mitigating the state space explosion commonly encountered in model checking. Experimental results demonstrate that our method achieves lower processor occupancy and improved optimization performance compared to traditional techniques, thereby enhancing both the practical applicability of partition scheduling in IMA systems and the overall efficiency of real-time system design.

Keywords Partition scheduling, Integrated modular avionics, Timed automata, UPPAAL, Genetic algorithm

The rapid advances in microprocessor technology and large-scale integrated circuits have significantly influenced the evolution of avionics systems. Modern commercial aircraft widely apply the Integrated Modular Avionics (IMA) architecture, which utilizes a limited number of general-purpose processor modules in place of a multitude of dedicated controllers¹. This transition enables the software implementation of a greater number of real-time avionics functionalities running on processors². Nonetheless, this approach results in the loss of physical isolation between avionics applications, thereby increasing the risk of fault propagation and system failures³. To address these challenges, the IMA systems employ partition scheduling to orchestrate the execution of diverse applications on a shared processor. The partitions are activated cyclically, based on a predefined scheduling table where each application operates exclusively within its designated partition, thus guaranteeing temporal isolation between applications⁴. Consequently, the partition scheduling largely determines both the temporal resource allocation and processor utilization in an IMA system.

As aircraft systems become more complex and mission-critical functions are increasingly consolidated into shared hardware platforms, the need for accurate and efficient partition scheduling has become more urgent. Ensuring that all partitions execute correctly without temporal interference is vital for flight safety and certification.

The goal of the partition scheduling problem in IMA systems is to acquire an optimal partition scheduling table in terms of processor utilization while guaranteeing the schedulability of all the partitions. Some studies simplify the system model by linearizing the schedulability constraints, solving the complex avionics scheduling problem based on classic mathematical optimization theories, such as linear programming⁵, mixed-integer optimization^{6–9}, and geometric programming¹⁰. Unfortunately, the worst-case assumptions of such simplified systems provide conservative schedulability conditions and aggravate the waste of processor time.

However, these techniques often require restrictive assumptions to make the problem analytically tractable. This leads to overly conservative results that waste processor time and hinder the system's overall performance. Moreover, these methods lack the expressiveness needed to capture the full semantics of two-level scheduling, preemption, and mixed-periodic partitions present in real IMA systems.

¹School of Computer Science, Northwestern Polytechnical University, Xi'an 710072, Shaanxi, China. ²College of Software Engineering, Zhengzhou University of Light Industry, Zhengzhou 450002, Henan, China. ✉email: zhaijun@nwpu.edu.cn; huangmin@zzuli.edu.cn

In contrast to traditional analytic scheduling methods, Model checking is a promising formal method that fully leverages the powerful expressiveness of formal models, enabling precise modeling of avionics systems and automated schedulability analysis. The formal model of Timed Automata (TA)¹¹ is representative for its rigorous syntax and temporal semantics, making it suited for describing the temporal behaviors of complex IMA systems. By exploring the state space of the TA model, the widely used model checker UPPAAL¹² can verify the schedulability property of the system in temporal logic expressions. The related work^{13–20} demonstrates highly precise schedulability bounds of the system model in UPPAAL.

Nevertheless, there are two main challenges in applying model checking to the partition scheduling of IMA systems. First, classic model checking is confronted with the state space explosion problem when handling complex system models. The model checker encounters a sharp exponential increase in the processing time and memory as more concurrent components are integrated into the system model. Second, the nature of the partition scheduling in an IMA system is a nonlinear non-convex optimization problem¹⁰, but model checking is a verification technique that does not easily describe or solve the optimization problem, which will lead to an inefficient exhaustive search for the optimal scheduling table in the system model.

These challenges point to a crucial gap in the current research landscape: the need for an integrated method that combines the accuracy and rigor of formal verification with the flexibility and efficiency of heuristic optimization. Bridging this gap can result in better scheduling quality, higher processor utilization, and greater system scalability – all of which are essential in modern avionics.

This paper proposes a model-based method of partition scheduling for IMA systems, which are modeled as timed automata in the model checker UPPAAL. The main contributions of this paper include:

1. *Novel Model-Based Partition Optimization*: The method introduces a novel model-based partition optimization technique for IMA systems compliant with the ARINC 653 avionics standard, effectively handling two-level scheduler, multicore processor, and both strictly and loosely periodic partitions. Unlike traditional analytical methods, this approach is more precise for system integrators as it leverages the powerful modeling capabilities of Timed automata in UPPAAL.
2. *Parallel Genetic Algorithm for Efficient Optimization*: The method incorporates a parallel genetic algorithm to explore the scheduling parameter space of the IMA model. This algorithm enables parallel processing of each partition and invokes the improved genetic operators during the evolution, accelerating the computationally expensive model-based analysis through parallelization and achieving an optimal solution more efficiently than traditional analytical methods.
3. *Compositional Verification Framework*: A new compositional verification framework is proposed that allows independent verification of each partition while combining local results to derive the global schedulability property. This innovative design significantly mitigates the state space explosion problem commonly encountered in model checking, enhancing the scalability and efficiency of the verification process.

Related work

The model-based approaches in related work primarily focus on two areas: schedulability analysis and scheduling system optimization. The schedulability analysis involves determining whether a system model with deterministic constraints (i.e., no unknowns) can meet its scheduling requirements. By contrast, the scheduling system optimization addresses models with unknown variables or parameters, seeking the optimal or feasible solution within the system's schedulability boundaries.

Schedulability analysis using model checking, particularly with Timed Automata (TA) and its extensions such as stopwatch automata, has been extensively researched since the 2000s due to its expressiveness and effectiveness in verifying real-time systems under strict timing constraints. Fersman et al.²¹ proposed a model of task automata, TA extended with real-time tasks triggered by events, for schedulability analysis of a single-processor real-time system, and implemented the verification tool TIMES based on UPPAAL. David et al.²² developed a model-based framework utilizing UPPAAL for schedulability analysis of embedded real-time systems, incorporating multi-processor configurations, task dependencies, and various scheduling policies. They proved that the schedulability can be checked for a subclass of timed automata but will be undecidable if the system model covers the execution time uncertainty and preemptive scheduling. However, the recent support for timed automata in UPPAAL leads to an efficient approximate analysis that has proved adequate on several concrete industries cases.

The approaches of scheduling system optimization typically rely on accurate and reliable schedulability analyses as their foundation. Boudjadar et al.^{13–15} proposed a compositional framework of parameterized stopwatch automata for analyzing hierarchical scheduling systems, and applied it to the schedulability analysis of an avionics system based on UPPAAL. They built and analyzed the model of each partition independently, estimating and optimizing its scheduling requirement by using Statistical Model Checking (SMC). Nevertheless, the combination of local optimization of individual partitions does not guarantee a globally optimal solution¹⁰. As a result, the scheduling parameters for each partition cannot be selected independently but jointly optimized.

Abdeddaim et al.^{23,24} presented an efficient shortest path algorithm for timed automata and stopwatch automata to find optimal schedules for the classical job-shop problem. They also used CTL model checking in UPPAAL to test the feasibility of a task set and its schedulability with Preemptive Fixed Priority (PFP) and Earliest Deadline First (EDF) policies. Le et al.²⁵ applied the TA formalism on a case study of heterogeneous communication systems, verifying the schedulability using UPPAAL, thereby exhaustively analyzing the properties of the system for every point in a grid of the design parameters. Glonina et al.²⁶ proposed a general stopwatch automata-based model of an IMA system, performing schedulability analysis of this general model instead of the corresponding concrete model, which is more efficient than model checking. This approach mandates the manual verification of a uncertain number of the correctness requirements to ensure the traces equivalence between models and

stipulates that the system models must be deterministic. Sun et al.²⁷ performed a component-based analysis of hierarchical scheduling systems using linear hybrid automata to improve the scalability of model checking. They also tentatively explored a grid of the design parameters and repeatedly tested the schedulability of the corresponding model, obtaining better resource utilization than analytical methods.

To further enhance the optimization of design parameters in system models, the Parametric Timed Automaton (PTA)²⁸ and its IMITATOR tool²⁹ were designed for parametric schedulability regions of complex real-time systems. Sun et al.^{30,31} applied PTA and IMITATOR to the parametric schedulability analysis of fixed-priority scheduling on a single-processor and distributed system, acquiring the smaller schedulability region of design parameters. André³² presented a PTA-based schedulability analysis of a simple real-time system on a single-processor, finding the feasible parameter zones via IMITATOR. This work was subsequently extended to a launcher flight control system to automatically synthesize the main parameters of this single-processor system³³. However, the optimization problem of partition scheduling in the IMA system addressed in our work involves determining an optimal partition scheduling table characterized by an uncertain number of time windows (parameters), rendering the problem unsuitable for direct modeling and analyzing using PTA.

Obviously, IMITATOR excels at handling systems with uncertain parameters by computing symbolically schedulable regions, making it particularly suitable for early design space exploration. This advantage builds on the inverse approach of parametric timed automata, which is able to synthesize parameter regions that preserve system behavior based on a given estimate³⁴. Its modeling expressiveness has been further enhanced by extensions that support parameter updates in clock distribution³⁵. Recently, minimum time synthesis techniques were introduced to optimize parameter values for improved timing efficiency^{36,37}. While these advances have significantly enhanced the modeling capabilities and analytical depth of the tool, some challenges remain when applied to the combinatorial scheduling structures in this paper. For example, in IMA systems, the number and configuration of partitioned time windows are often unknown and need to be optimized. Capturing this variability directly in PTA models is inherently complex, especially in the presence of mixed partition cycles and multicore architectures.

In contrast, UPPAAL uses non-parametric timed automata, providing an efficient support for verifying schedulability under fixed parameters. In³⁸, a genetic algorithm was employed to explore the scheduling parameter space of an IMA system, invoking UPPAAL to verify the schedulability of each candidate solution. This approach relied on conventional genetic operators, whose configurations had a significant influence on the optimization outcomes. To improve the operators' configuration, a Q-learning mechanism was further introduced in³⁹, enabling the adaptive configuration of genetic operators based on the search history. However, these studies did not effectively incorporate domain-specific knowledge to guide the design of genetic operators, which limited the search efficiency for large systems. Additionally, the generation of scheduling tables for mix-periodic partitions was not considered, potentially making the approaches impractical in a real IMA system and leading to inefficient processor utilization.

In recent years, a number of scheduling design and configuration approaches have been proposed for IMA systems. Cheptsov and Khoroshilov⁴⁰ proposed a robust resource partitioning method for ARINC 653 RTOSs using static Memory Management Unit(MMU) configuration. Zuepke et al.⁴¹ presented a unified AUTOSAR-OS and ARINC 653 kernel to combine avionics safety with the resource-efficiency of automotive systems. Lukic et al.⁴² proposed an automated approach for generating and verifying ARINC 653 configuration files, which reduces manual errors and improves safety assurance. In parallel, Zhang et al.⁴³ applied refinement-based formal specification to analyze the behavior of ARINC 653 systems on multicore architectures. These studies further support the integration of formal methods into IMA system design and configuration.

In this paper, we explore how model-based methods can be employed to solve the optimization problem for IMA partition scheduling systems, aiming to find the optimal partition scheduling table with respect to processor utilization.

Preliminaries

This section introduces the definition of timed automata, including its syntax, the extension of stopwatches and a simple example in the UPPAAL tool, which helps us understand the model of timed automata intuitively.

Definition of timed automata

Timed Automata are formal models that extend traditional finite-state automata by incorporating the notion of time. They are particularly useful for modeling systems that exhibit both discrete (event-driven) and continuous (time-driven) behaviors. A timed automaton operates in a state space where transitions between states can be triggered by both the occurrence of events and the expiration of time intervals.

A timed automaton¹¹ can be formally defined as a tuple $A = \langle L, \Sigma, X, E, I, l_0, F \rangle$ where:

1. L is a finite set of locations. Each location represents a set of states of the system being modeled.
2. Σ is a finite set of actions that can trigger transitions between locations.
3. X is a finite set of clock variables. These variables are used to measure the passage of time and can be reset to zero at specific transitions.
4. E is a set of edges between locations. Each edge $e \in E$ representing a set of transitions is a tuple $\langle l, g, a, \mu, l' \rangle$ where:
 - $l, l' \in L$ are the source and target locations, respectively.
 - g is a guard condition over the clock variables X . It is a linear constraint that specifies when a transition can occur (, i.e. what time constraints must be met).

- $a \in \Sigma \cup \epsilon$ is the action associated with the transitions. If $a = \epsilon$, the transition is classified as spontaneous and internal, thereby indicating that it is neither triggered by nor results in any external event.
 - μ is a clock reset function that specifies which clocks should be reset to zero upon taking the transitions.
5. I is an invariant function that maps each location to a linear constraint (invariant) that must be held continuously at the location.
 6. l_0 is an initial location. It defines the starting point of the timed automaton's execution.
 7. F (optional) is the set of accepting (or final) locations, which mark successful termination or reachability of desired states in the modeled system.

A timed automaton evolves over time by advancing its clock variables continuously and making transitions based on the expiration of time intervals (as specified by guard conditions) and the occurrence of actions. This combination of discrete and continuous behaviors makes timed automata a powerful tool for modeling and verifying real-time systems.

In the context of timed automata, stopwatches are a special type of clock variables that can be paused and resumed, unlike a regular clock that continuously measures time. Specifically, a stopwatch's timing can be halted and restarted based on changes in the system's state. This feature makes it particularly useful for modeling and analyzing certain systems, such as preemptive scheduling systems.

The primary purpose of using a stopwatch in timed automata is to provide more precise control and description of system behavior. It allows for more accurate modeling of processes that start and pause, such as simulating the time consumption of a task when it is interrupted and then resumed. In practical applications, a stopwatch can help timed automata achieve stricter time constraints and stronger expressiveness.

Timed automata in UPPAAL

UPPAAL is an integrated tool designed for modeling and verifying timed automata, implementing symbolic model checking for Timed Computation Tree Logic (TCTL) expressions. The tool supports graphical representation of timed automata, facilitating intuitive and efficient system modeling. Each individual automaton structure is referred to as a *template*⁴⁴.

Figure 1 presents a simple timed automaton template that includes the four types of locations defined by UPPAAL. The double circular *InitialLocation* serves as the starting position of the system. The circular *NormalLocation* represents ordinary states, illustrating the operational states of the system within a time interval. The *UrgentLocation*, marked by a “U” symbol, defines urgent states where UPPAAL disallows the passage of time. The committed locations, denoted by a “C” symbol in *CommittedLocation*, not only freeze the passage of time but also require that the subsequent transition must originate from one of them, which is useful for establishing atomic operations or conditional branching structures.

The linear constraints such as $i \leq 5$ and $i' == 0$ act as invariants for their respective locations, defining the conditions when the system resides in such a location. UPPAAL utilizes invariants to support stopwatch variables, as illustrated by the invariant $i' == 0$ at the *NormalLocation*, indicating that the derivative of the clock variable i is zero, thus preventing any changes to i at this location.

The edges connecting locations are annotated with four types of specifications: *selection*, *guard*, *channel*, and *update*. The *selection* annotation $t : type$ binds a nondeterministic value from the *type* to the identifier t , thereby reducing the number of edges between identical locations. The linear constraint $i \geq 5$ serves as a *guard* for this edge, enabling the transition only when the condition is satisfied. The notation $channels[t]$ represents the external actions of a timed automaton, where the suffixes “!” and “?” indicate output and input *channels* respectively; an edge marked with an output channel $c!$ synchronizes with another edge labeled with an input channel $c?$; in cases where an edge lacks an explicit channel, the transition is executed asynchronously within the automaton. The expression $i = 0$ represents an *update* operation associated with the edge, allowing for the resetting of clocks like i and modification of integer variable values during transitions.

UPPAAL uses the TCTL formula $A[\Box]\varphi$ to describe safety properties of timed automata. In this formula, “A” represents All (all paths), “[$\Box$]” represents Globally (All states on the path), and φ is a state formula that defines the desired states without considering the behavior of the model. UPPAAL verifies a safety property by exploring the reachability of its faulty states within the symbolic state space of the model, thereby determining whether the model satisfies the safety property.

However, it has been proved that the reachability problem is undecidable for the verification of timed automata including stopwatches (stopwatch automata)⁴⁵. As a result, UPPAAL offers an approximate reachability algorithm for stopwatch automata. This algorithm performs only reachability analysis and yields an over-approximation of the reachable state-space: If UPPAAL indicates that a particular state is not reachable, it is

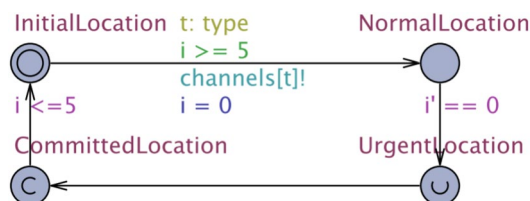


Figure 1. A template of timed automata in UPPAAL.

definitively unreachable. Conversely, if UPPAAL returns the result “may (may not) be satisfied”, this suggests that the identified trace may be spurious, indicating uncertainty regarding the satisfaction of the specified property.

The over-approximation is not precise but still suitable for safety property-checking like the step of schedulability verification in this paper. In our stopwatch automata model, an *Unschedulable* location is defined for error states that any task misses the deadline. Thus the schedulability verification is also a reachability problem of the error states. If we obtain the result “Satisfied (unreachable)” from UPPAAL, it means the error states are definitely unreachable and the system is absolutely schedulable. Therefore, we can acquire the approximate schedulability bounds of the system based on the positive results of model checking.

Furthermore, in a partitioned system, multiple tasks are encapsulated within a partition automaton. The partition scheduling is modeled at a higher level using another timed automaton that cyclically activates each partition according to the scheduling table. This hierarchical modeling reflects the two-level scheduling mechanism defined in ARINC 653.

Problem description

This section first provides the formal definition of the IMA scheduling system and subsequently describes its partition scheduling as an optimization problem.

System definition

According to the ARINC 653 standard⁴, we consider an IMA operating system running on a single-core or multi-core processor with the core set $\mathcal{C}$. As shown in Fig. 2, it is a two-layer partitioned scheduling system that consists of the global and local layer. In the global layer, partitions are activated by a Time Division Multiplexing (TDM) global scheduler across all processor cores synchronously⁴⁶. For each partition in the local layer, there is a local scheduler employing the preemptive Fixed Priority (FP) policy to schedule the execution of internal real-time tasks.

The system is divided into n temporal partitions $\Omega = \{\mathcal{P}_i \mid i = 1, 2, \dots, n\}$, where $\mathcal{P}_i$ denotes the i th partition of the system. The temporal partitions are scheduled by the TDM global scheduler repeatedly every major time frame M in accordance with a partition scheduling table $\mathcal{S} = \{W_i \mid i = 1, 2, \dots, n\}$, where $W_i = \{w_k^i \mid k = 1, 2, \dots, \omega_i\}$ is the set of $\mathcal{P}_i$'s partition time windows (the time slices of $\mathcal{P}_i$). The k th partition time window of $\mathcal{P}_i$ is defined as $w_k^i = \langle o_k^i, d_k^i \rangle$, where o_k^i represents the offset from the start of the major time frame, and $d_k^i > 0$ denotes the window's duration. The time windows of partitions are non-overlapping,

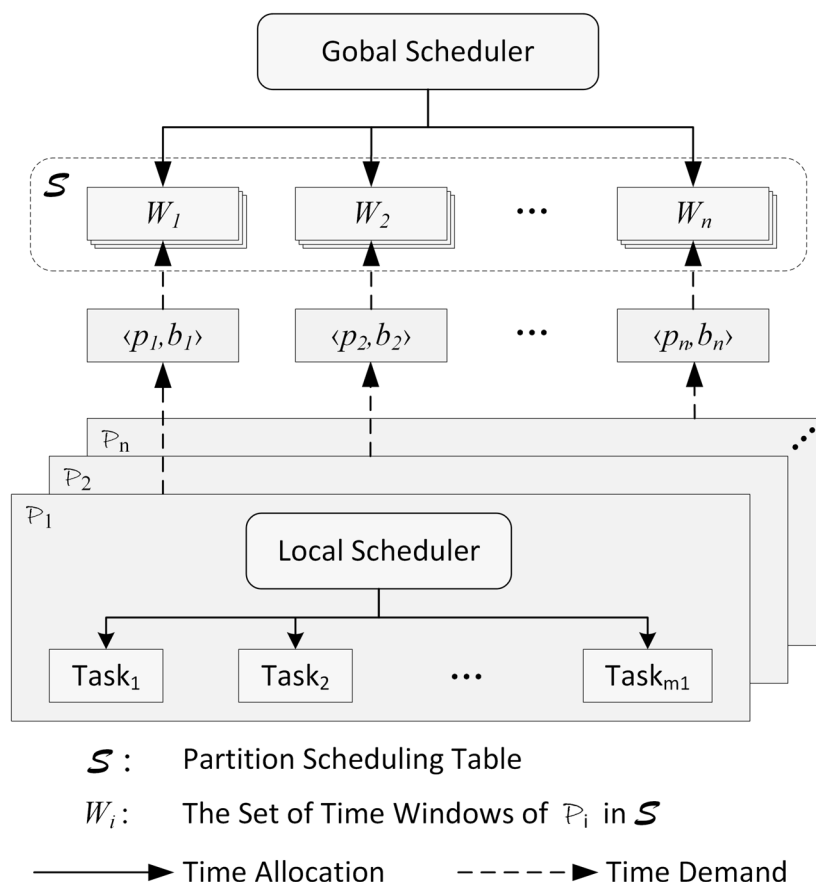


Figure 2. The partition scheduling system in IMA architecture.

thus maintaining the absolute temporal isolation between partitions. For any two partition time windows $w_k^i = \langle o_k^i, d_k^i \rangle$ and $w_l^j = \langle o_l^j, d_l^j \rangle$ if $i \neq j$ or $k \neq l$ then we have $o_k^i + d_k^i \leq o_l^j$ or $o_l^j + d_l^j \leq o_k^i$. There exists the context-switching overhead between partitions. It involves saving and restoring processor state and switching between partitions, mainly depending on the implementation of related hardware. Since the ARINC 653-compliant system requires a predictable temporal behavior, we assume the context-switching overhead to be a system-level constant v .

For any partition $\mathcal{P}_i$, there is a real-time task set $\Gamma_i = \{\tau_j^i \mid j = 1, 2, \dots, m_i\}$, where the tasks within $\mathcal{P}_i$ can only be scheduled or executed within the partition time windows of W_i . The task scheduling policy within each partition is independent of the system's partition scheduling strategy. A task τ_j^i is defined as a tuple $\langle T_j^i, I_j^i, E_j^i, G_j^i, D_j^i, R_j^i, C_j^i \rangle$. T_j^i is the period of task, I_j^i is the initial offset, E_j^i is the minimum execution time in a period, G_j^i means the maximum, D_j^i denotes the relative deadline, R_j^i denotes the priority, and $C_j^i \in \mathcal{C}$ represents the processor core that τ_j^i is bound.

Each task considers a flexible execution interval, defined by the minimum and maximum execution times (E_j^i, G_j^i), reflecting potential runtime variations due to conditional execution paths, cache behavior, memory contention and other hardware-level fluctuations. Additionally, the processor affinity C_j^i clearly associates each task with a specific processing core, which is critical for accurately modeling task execution behaviors and interference patterns in multicore avionics platforms. This precise characterization of task parameters provides a rigorous foundation for evaluating partition schedulability and optimizing overall processor utilization.

In this system, the scheduling table $\mathcal{S}$ stipulates the time allocation to all the partitions. The ARINC 653 standard⁴ defines a pair of scheduling parameters $\langle p_i, b_i \rangle$ as the periodic time demand of a partition $\mathcal{P}_i$. The parameter p_i denotes a partition period and b_i is the budget every p_i . Thus the major time frame M , the hyper-period of all partitions, is typically defined as the least common multiple of all partition periods. The schedulability condition of the system is guaranteed if and only if the time allocation always satisfies the time demand of each partition, where all the internal tasks meet their deadlines.

In the scheduling table $\mathcal{S}$, there are two types of temporal partitions that conform to the periodic time demand⁴⁷. The first is strictly periodic partitions that support the strict periodicity of tasks in the partition. As illustrated in Fig. 3a, in strictly periodic partitions, there is only one continuous time window within each period, and the offset of each window is identical. The second is loosely periodic partitions, providing a “best-effort” periodic time service. Although a loosely periodic partition $\mathcal{P}_i$ still achieves its periodic time demand $\langle p_i, b_i \rangle$, it allows for a certain amount of jitter of the windows. Additionally, each period may contain multiple time windows. For example, in Fig. 3b, a loosely periodic partition $\mathcal{P}_i$ has one window in the first period but two in the second, where $d_1^i = d_2^i + d_3^i$ and $o_1^i \neq o_2^i$.

The ARINC 653 standard defines a partition scheduling mechanism that allocates fixed-duration time windows to each partition at regular intervals. From the perspective of hierarchical scheduling⁴⁸, this behavior is functionally equivalent to a periodic server, where the partition receives a fixed execution budget every period and unused time does not carry over. Compared to more flexible server strategies such as deferrable servers (which allow unused capacity to be deferred within the same period) or sporadic servers (which support event-driven activations with minimum inter-arrival constraints), ARINC 653 enforces strict periodicity and deterministic execution slots. These constraints enhance analyzability and isolation, which are critical in avionics applications. Although only periodic partition behavior is considered in this work, the modeling framework can be extended to support other server types by modifying partition activation and budget replenishment logic.

Partition scheduling optimization problem

The partition scheduling optimization problem of an IMA system can be defined as follows: Given the above partition scheduling system $\Omega = \{\mathcal{P}_i \mid i = 1, 2, \dots, n\}$, the problem is to find a partition scheduling table $\mathcal{S} = \{W_i \mid i = 1, 2, \dots, n\}$ such that all the tasks τ_j^i in any partition $\mathcal{P}_i$ satisfy their deadline constraints, while minimizing the processor occupancy:

$$U_{min} = \min \left\{ \sum_{i=1}^n \sum_{k=1}^{\omega_i} \frac{d_k^i + v}{M} \right\} \quad (1)$$

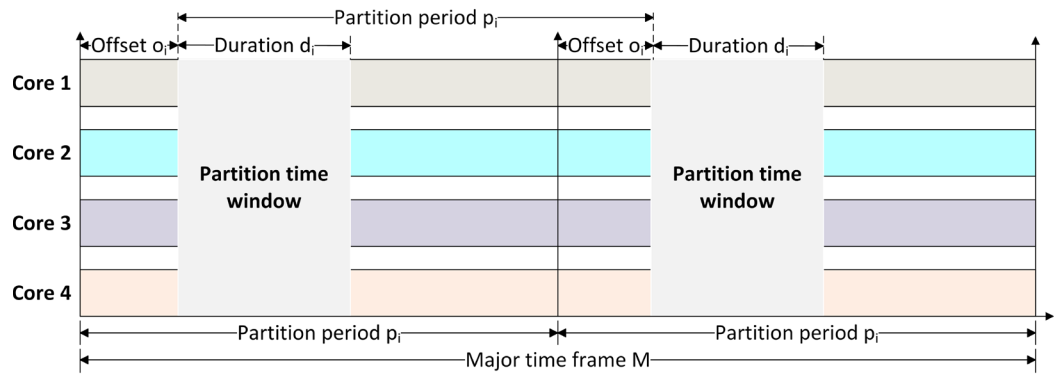
where v represents the overhead for each partition context switch.

Owing to the unknown number ω_i of partition time windows, it is difficult to solve the objective function in Eq. (1). Consider a balance between the temporal supply of the partition time windows W_i in schedule $\mathcal{S}$ and the demand for the periodic scheduling parameters $\langle p_i, b_i \rangle$. We use the scheduling parameters $\langle p_i, b_i \rangle$ as the unknown variables instead of the scheduling table $\mathcal{S}$.

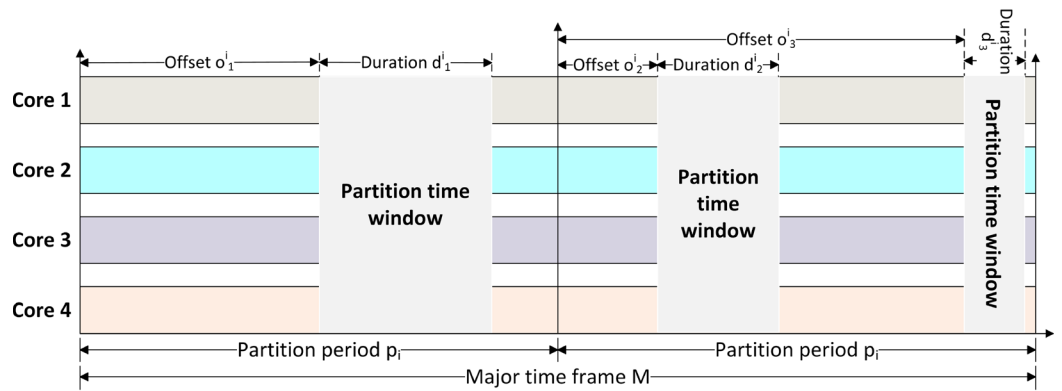
Since the major time frame M is the least common multiple of all partition periods p_i , we assume that $M = k_i p_i, k_i \in \mathbb{N}_+$. Thus, it holds that

$$\sum_{k=1}^{\omega_i} d_k^i = k_i b_i \quad (2)$$

Let c_j^i be the number of context switches of the partition $\mathcal{P}_i$ during the j th period of M . We define the processor occupancy:



(a) Strictly periodic partition



(b) Loosely periodic partition

Figure 3. An example of two types of temporal partitions.

$$\begin{aligned}
 U &= \sum_{i=1}^n \sum_{k=1}^{\omega_i} \frac{d_k^i + v}{M} \\
 &= \sum_{i=1}^n \frac{1}{k_i p_i} \left(k_i b_i + \sum_{j=1}^{k_i} c_j^i v \right) \\
 &= \sum_{i=1}^n \frac{1}{p_i} \left(b_i + \sum_{j=1}^{k_i} \frac{c_j^i}{k_i} v \right) \\
 &= \sum_{i=1}^n \frac{b_i + c_i v}{p_i}
 \end{aligned} \tag{3}$$

where c_i is the average number of context switches for the partition $\mathcal{P}_i$.

Therefore, the partition scheduling optimization problem involves finding a $2n$ -dimensional vector $x = (x_1, x_2, \dots, x_{2n})^T \in \mathbb{R}_+^{2n}$ build the periodic scheduling parameters $\langle p_i, b_i \rangle$ of the partition $\mathcal{P}_i$, such that all the tasks τ_j^i in $\mathcal{P}_i$ satisfy their deadline constraints, while minimizing the processor occupancy:

$$U_{min} = \min \left\{ \sum_{i=1}^n \frac{b_i + c_i v}{p_i} \right\} \tag{4}$$

Since the vector x of unknown variables describes the temporal demand of all the partitions, we call x the demand vector of the system.

Optimization method

To address the optimization problem defined in “[Problem description](#)”, this work proposes a model-based partition scheduling framework that integrates heuristic optimization and formal verification. The goal is to construct a feasible and efficient partition scheduling table under multiple system-level constraints, such as partition types, processor binding, and execution time bounds. The proposed method formulates the scheduling table generation as a combinatorial optimization problem, where candidate solutions are encoded as scheduling parameters. A genetic algorithm is used to iteratively explore the solution space, guided by a fitness function derived from the schedulability evaluation. The verification of schedulability is performed through

compositional model checking, where each partition is independently modeled and analyzed using UPPAAL. The overall mechanism is illustrated in Fig. 4.

The IMA partition scheduling system is modeled as timed automaton templates in UPPAAL. The users of this optimizer, IMA integrators, employing the UPPAAL templates $\mathcal{M}$ along with the system configuration as input, start the optimization process at generation $g = 1$ and terminate at $g = G$ to identify the optimal partition scheduling table $\mathcal{S}_{opt}$. The optimizer invokes a parallel genetic algorithm to explore the scheduling parameter space of the IMA model. The scheduling parameters of the system are encoded as an individual vector in polar coordinates, which can be translated by a periodic demand decoder into a demand vector. For each individual I in the population $\beta_g, g \in \mathbb{N}_+$ at the g th generation, a candidate partition scheduling table $\mathcal{S}$ is generated according to its demand vector x . By applying a compositional approach, the optimizer checks the schedulability properties of both the strictly and loosely periodic partitions in an IMA system. Each of the partitions is processed independently by model checking during the evolution. A fitness function evaluates every individual in a population on the basis of all the schedulability conclusions of a system, finally outputting the optimal solutions or providing the fitness values for the next generation.

The optimizer consists of the following components:

- (1) Parallel genetic algorithm The optimizer employs a parallel genetic algorithm that explores the solution space of the system to find an optimal partition scheduling table. The genetic algorithm randomly initializes the first generation of a population $\beta_1 = \{I_k^1 \mid k = 1, 2, \dots, K\}$ with K individual vectors

$$I_k^1 = (\alpha_1^{(1,k)}, \alpha_2^{(1,k)}, \dots, \alpha_{2n}^{(1,k)})^T. \text{ A constant } G \text{ is predefined as a stopping criterion, indicating the maximum number of generations in the genetic algorithm.}$$

For each generation $g \in \{1, 2, \dots, G\}$, genetic operators are invoked to recombine and mutate individuals in the population, generating the next generation of the population β_{g+1} . The individuals in a population are evaluated simultaneously via model checking. Finally, after G generations of evolution, the optimizer selects the optimal individual vector and its partition scheduling table $\mathcal{S}_{opt}$ with the minimum processor occupancy U_{min} .

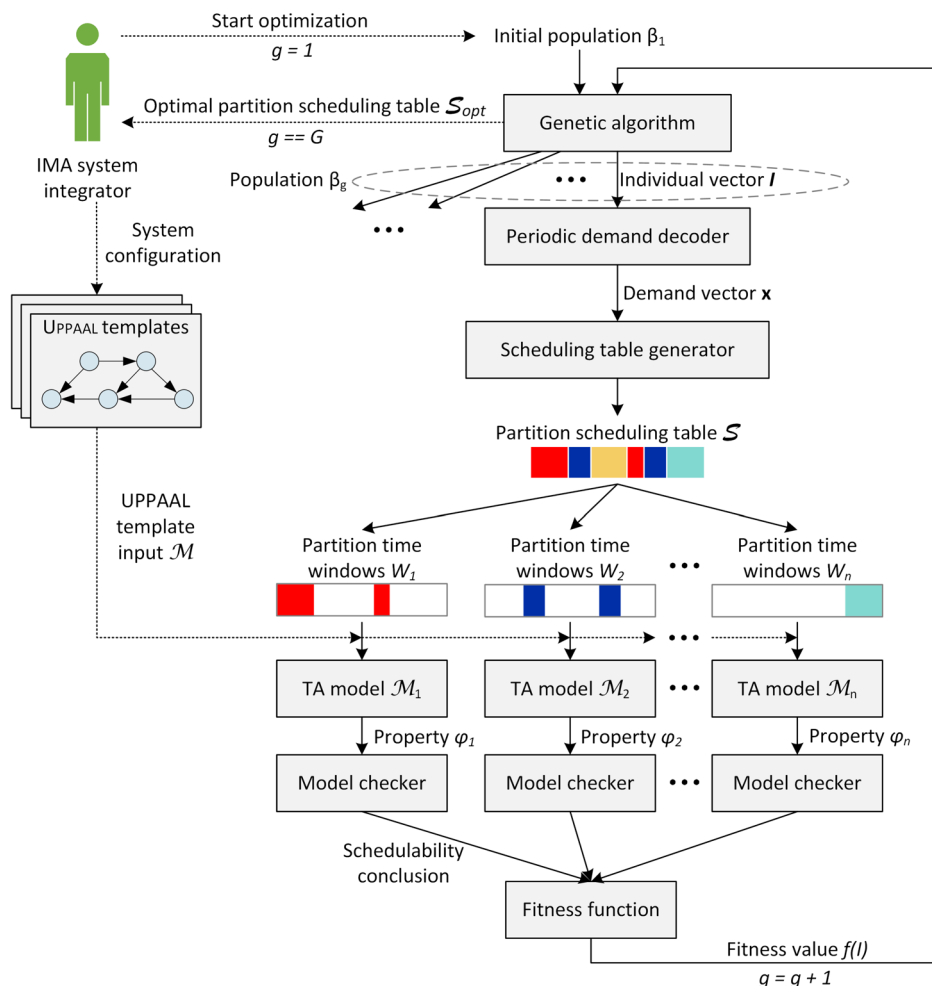


Figure 4. Data flow of the model-based optimizer for the partition scheduling problem.

- (2) **Periodic demand decoder** The period demand decoder realizes the transformation from the individual vector $I = (\alpha_1, \alpha_2, \dots, \alpha_{2n})^T$ to the demand vector $x = (x_1, x_2, \dots, x_{2n})^T$. In the optimizer, the parallel genetic algorithm manipulates the encoded individual vectors instead of the demand vectors. A demand vector is encoded into an individual vector in polar coordinates according to the rules:

$$\alpha_{2i-1} = \arctan(x_{2i}/x_{2i-1}), \alpha_{2i} = \sqrt{x_{2i-1}^2 + x_{2i}^2} \quad (5)$$

which helps extract the similarities from selected individuals in the recombination operation of the genetic algorithm. This encoding approach enhances the search efficiency by improving the survival rate of offspring individuals in the evolution. The design of the genetic operators is presented in “[Design of the genetic operators](#)”.

- (3) **Scheduling table generator** The scheduling table generator produces a partition scheduling table $S = \{W_i \mid i = 1, 2, \dots, n\}$ based on the demand vector x , handling two types of temporal partitions: the strictly periodic partitions, ensuring strict periodicity of the time slots for each partition, and the loosely periodic partitions, providing a “best-effort” periodic time service. Different generation methods are employed for the two types of partitions. The strictly periodic partitions are first settled by using mixed integer optimization. Subsequently, the loosely periodic partitions are scheduled in order of their jitter constraints. The method of how to generate the scheduling table is detailed in “[Generation of partition scheduling table](#)”.
- (4) **Compositional model checking** We apply a compositional approach to the schedulability verification of the system model in timed automata, which is equipped with an input partition scheduling table S . Since the n partitions in the system are totally isolated, their temporal behaviors can be checked simultaneously and thus their schedulability properties are concluded independently of each other. Finally, the global schedulability is derived from the n local positive results of all the partitions, thereby alleviating the state space explosion of verifying a monolithic system model. We present the modeling framework of a system and its compositional approach in “[Formal model of IMA scheduling systems](#)”.
- (5) **Fitness function** The fitness function is based on the results of model checking, evaluating the fitness value of an individual vector I corresponding to a partition scheduling table S . The fitness values guide the genetic algorithm in exploring the solution space of the system’s partition scheduling table. A higher fitness value of an individual indicates its higher quality, increasing the probability of its features being retained in the next generation. The fitness values are assigned according to the number of schedulable partitions and processor utilization. Specifically, individuals with more schedulable partitions receive higher fitness values. For individuals with the same number of schedulable partitions, those who have lower processor occupancy are assigned higher fitness values. The fitness function for individual I is defined as follows:

$$f(I) = rN + r \left(1 - \sum_{i=1}^n \sum_{k=1}^{\omega_i} \frac{d_k^i + v}{M} \right) \quad (6)$$

where r is a user-defined scaling factor (e.g. 100) that allows us to proportionally convert the decimal form of fitness values into integers, N denotes the number of the partitions that go through the schedulability verification via model checking and $\sum_{i=1}^n \sum_{k=1}^{\omega_i} \frac{d_k^i + v}{M}$ represents the processor utilization of the system.

Algorithm 1 provides the pseudo-code for the model-based optimization method. As shown in Algorithm 1, the users input the set of timed automation templates including system configuration, finally obtaining the optimal scheduling table of the system.

Input: $\mathcal{M}$, the templates of system model

Output: $\mathcal{S}_{out}$, the output partition scheduling table

```

1: Initialize a population  $\beta_1 = \left\{ \left( \alpha_1^{(1,k)}, \alpha_2^{(1,k)}, \dots, \alpha_{2n}^{(1,k)} \right)^T \mid k = 1, 2, \dots, K \right\}$ 
2: for  $g = 1$  to  $G$  do
3:   for  $k = 1$  to  $K$  par-do
4:     Generate demand vector  $x_k^g = \left( x_1^{(g,k)}, x_2^{(g,k)}, \dots, x_{2n}^{(g,k)} \right)^T$  by decoding the
       individual  $I_k^g = \left( \alpha_1^{(g,k)}, \alpha_2^{(g,k)}, \dots, \alpha_{2n}^{(g,k)} \right)^T$  in  $\beta_g = \{I_1^g, I_2^g, \dots, I_K^g\}$ 
5:   Generate scheduling table  $\mathcal{S}_k^g$  based on the demand vector  $x_k^g$ 
6:   for  $i = 1$  to  $n$  par-do
7:     Instantiate the model  $\mathcal{M}_i$  of partition  $\mathcal{P}_i$  by importing  $\mathcal{S}_k^g$  into  $\mathcal{M}$ 
8:     Perform model checking of  $\mathcal{M}_i$  for the schedulability of  $\mathcal{P}_i$ 
9:   end for
10:  Evaluate the fitness  $f(I_k^g)$  based on the schedulability of  $n$  partitions
11: end for
12: if  $g < G$  then
13:    $\hat{\beta}_g \leftarrow selectionOperator(\beta_g)$ 
14:    $\bar{\beta}_g \leftarrow recombinationOperator(\hat{\beta}_g)$ 
15:    $\beta_{g+1} \leftarrow mutationOperator(\bar{\beta}_g)$ 
16: end if
17: end for
18: return the partition scheduling table  $\mathcal{S}_{out}$  with the best fitness value.

```

Algorithm 1. Model-based optimization of partition scheduling

The optimizer begins by randomly initializing a population in Line 1.

Lines 2 to 17 delineate the iterative process of the genetic algorithm across G generations. For each generation, the algorithm concurrently evaluates all K individuals in the population by utilizing the keyword “par-do” from Line 3 to Line 11. Each partition is further processed in parallel (Lines 6 to 9) through model checking in UPPAAL, which represents the most computationally intensive operation during the evolution. This substantial degree of parallelization effectively reduces the model size during the model checking phase, thereby alleviating the state space explosion typically associated with schedulability verification.

Line 10 gathers the schedulability results of n partitions in the k th individual I_k^g and subsequently assesses the fitness value of I_k^g .

Lines 12 to 16 detail the reproductive operations within the genetic algorithm for generation 1 through $G - 1$. The population β_g undergoes selection, recombination, and mutation processes to generate the subsequent population β_{g+1} .

Ultimately, the optimizer outputs the optimal partition scheduling table $\mathcal{S}_{opt}$, which encapsulates the highest fitness value identified throughout the evolutionary process.

Design of the genetic operators

In the genetic algorithm, a population $\beta_g = \{I_k^g \mid k = 1, 2, \dots, K\}$ is processed by the three genetic operators: selection, recombination and mutation operator in sequence, finally producing the next generation of the population $\beta_{g+1} = \{I_k^{g+1} \mid k = 1, 2, \dots, K\}$.

Selection operator

The selection operator is to fetch K pairs of individuals from the population β_g according to the fitness evaluation of all the individuals. During the evolution, it is highly probable to generate low-fitness individuals, such as invalid demand vectors whose processor occupancy exceed 100%. Therefore, we adopt an exponential ranking selection operator, which applies higher selective pressure and has a stronger tendency to choose better individuals from the population, while maintaining a certain standard deviation of the fitness distribution after selection.

In the exponential ranking selection, the K individuals in β_g are first ranked based on the fitness values from the worst (rank 1) to the best (rank K). The probability ρ_i of selecting the i th individual is calculated using the following formula²⁰:

$$\rho_i = \frac{c^{K-i}}{\sum_{k=1}^K c^{K-k}} \quad (7)$$

where $c \in (0, 1)$ adjusts the selective pressure of the operation. A smaller c increases the selective pressure, making the best-fit individuals more likely to be selected. The K pairs of individuals selected by the operator will be used as K pairs of parents to generate the offspring population β_{g+1} .

Recombination operator

The recombination operator is to extract the similarities from selected individuals during the evolution⁴⁹. We use the individual vectors in polar coordinates instead of the demand vectors to recombine the information from parents about the period p_i and budget b_i for each partition $\mathcal{P}_i (i = 1, 2, \dots, n)$, which retains similarity in the utilization ratio $u_i = b_i/p_i$.

Let $I_1^g = (\alpha_1^{(g,1)}, \alpha_2^{(g,1)}, \dots, \alpha_{2n}^{(g,1)})^T$ and $I_2^g = (\alpha_1^{(g,2)}, \alpha_2^{(g,2)}, \dots, \alpha_{2n}^{(g,2)})^T$ be a pair of selected individuals as parents at the g th generation. An intermediate recombination operator is utilized to generate an offspring individual $\bar{I}_1^{g+1} = (\bar{\alpha}_1^{(g+1,1)}, \bar{\alpha}_2^{(g+1,1)}, \dots, \bar{\alpha}_{2n}^{(g+1,1)})^T$ using the following rule:

$$\bar{\alpha}_k^{(g+1,1)} = \alpha_k^{(g,1)} + \lambda_k (\alpha_k^{(g,2)} - \alpha_k^{(g,1)}), \quad k = 1, 2, \dots, 2n \quad (8)$$

where $\lambda_k \sim U(-d, 1+d)$ denotes a scaling factor selected uniformly at random over an interval $[-d, 1+d]$ for each parameter.

Figure 5 shows an example of such a recombination operation. Let $x = (x_1, x_2, \dots, x_{2n})^T$ and $y = (y_1, y_2, \dots, y_{2n})^T$ be the demand vector of two parent individuals I_1^g and I_2^g respectively. Consider the partition $\mathcal{P}_i$. Its period p_i is depicted as X -axis and its budget b_i as Y -axis. Based on the encoding rules in Eq. (5), the individual parameters $\alpha_{2i-1}^{(g,1)} = \theta_x = \arctan(x_{2i}/x_{2i-1})$, $\alpha_{2i-1}^{(g,2)} = \theta_y = \arctan(y_{2i}/y_{2i-1})$ are slope angles and $\alpha_{2i}^{(g,1)} = l_x = \sqrt{x_{2i-1}^2 + x_{2i}^2}$, $\alpha_{2i}^{(g,2)} = l_y = \sqrt{y_{2i-1}^2 + y_{2i}^2}$ are Euclidean distance. As shown in Fig. 5, the gray sector is the value range of the offspring generated by the parent individuals. They combine similar features and generate a child individual $\bar{I}_1^{g+1}$ whose demand vector is decoded into $z = (z_1, z_2, \dots, z_{2n})^T$.

Mutation operator

The mutation operator provides random genetic variation to keep a certain degree of population diversity. Assume that our mutation operator acquires an offspring individual $\bar{I}_1^{g+1} = (\bar{\alpha}_1^{(g+1,1)}, \bar{\alpha}_2^{(g+1,1)}, \dots, \bar{\alpha}_{2n}^{(g+1,1)})^T$ after a recombination operation. A new individual $I_1^{g+1} = (\alpha_1^{(g+1,1)}, \alpha_2^{(g+1,1)}, \dots, \alpha_{2n}^{(g+1,1)})^T$ is computed according to

$$\alpha_k^{(g+1,1)} = \bar{\alpha}_k^{(g+1,1)} + \xi_{[k/2]} \Delta_k, \quad k = 1, 2, \dots, 2n \quad (9)$$

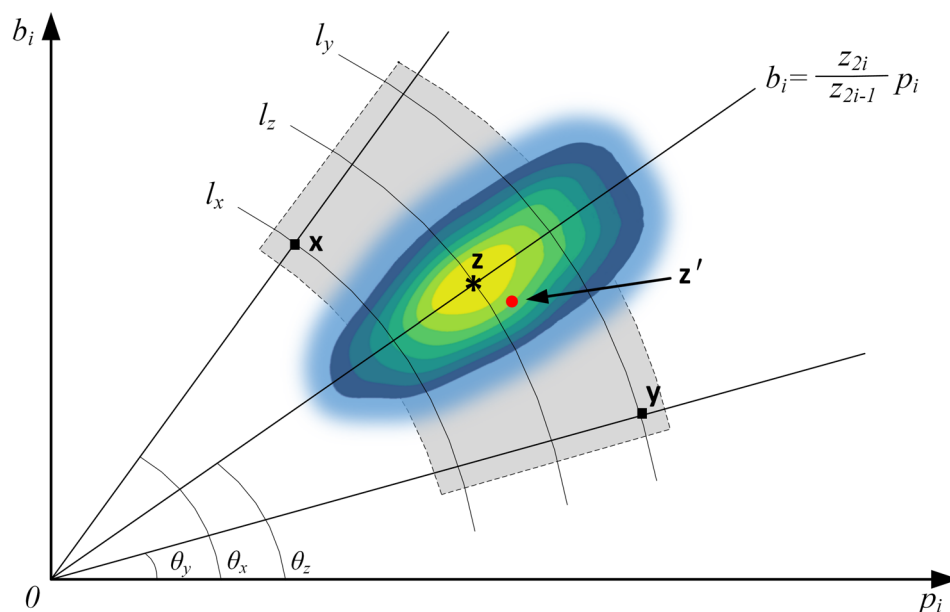


Figure 5. An example of a recombination and mutation operation.

where $\xi_{\lceil k/2 \rceil} \sim B(1, 1/n)$ is a mutation decision sampled from a Bernoulli distribution $B(1, 1/n)$ and $\Delta_k \sim N(0, \sigma_k)$ is a random sample from a normal distribution $N(0, \sigma_k)$.

The two parameters of a partition $\mathcal{P}_{\lceil k/2 \rceil}$ are chosen with the probability $1/n$ for mutation operation. Each of the variables is mutated independently with the offset Δ_k . Hence at least one partition is commonly mutated and prefers small offset during mutation to avoid a large number of invalid or non-schedulable time demands.

Figure 5 also depicts the offspring demand vector z' of a partition $\mathcal{P}_i$ after mutating the individual of z . The probability density contours of normal distribution $N(0, \sigma_k)$ in polar coordinates are deformed to avoid lines (taper at the origin) in cartesian coordinate system.

Generation of partition scheduling table

According to the demand vector x of an offspring individual, the scheduling table generator produces a partition scheduling table $\mathcal{S} = \{W_i \mid i = 1, 2, \dots, n\}$, handling both the strictly and loosely periodic partitions. We use different generation methods for them. First, the strictly periodic partitions are settled by using mixed integer optimization. Subsequently, the loosely periodic partitions are scheduled in order of the upper bounds of their jitters.

Scheduling strictly periodic partitions

We consider the set of strictly periodic partitions $\Omega_1 = \{\mathcal{P}_i \mid i = 1, 2, \dots, n_1\}$ in the system. Let $W_i = \{w_k^i \mid k = 1, 2, \dots, \omega_i\}$ be the set of $\mathcal{P}_i$'s partition time windows and $w_k^i = \langle o_k^i, d_k^i \rangle$ the k -th window.

For each strictly periodic partition $\mathcal{P}_i$, there is only one window $\langle o_k^i, d_k^i \rangle$ within the k th period of a major time frame M . Each window of $\mathcal{P}_i$ has the same offset o_i from the start of its period and the identical duration d_i . Thus, the scheduling tables $\mathcal{S}_i = \{W_i\}$ of $\mathcal{P}_i$ are generated based on this periodic time window $\langle o_i, d_i \rangle$.

Considering the periodic time demand $\langle p_i, b_i \rangle$ in the demand vector x , we acquire the duration $d_i = b_i$, and the offset $o_i \in O$ is determined by the following linear programming method.

Assume that two different partitions $\mathcal{P}_i$ and $\mathcal{P}_j$ have the strictly periodic time window $\langle o_i, d_i \rangle$ and $\langle o_j, d_j \rangle$ respectively. Since any two different partitions must be non-overlapping on the same processor, we can obtain the following inequality according to the conditions in⁵⁰:

$$d_i \leq (o_j - o_i) \bmod g_{i,j} \leq g_{i,j} - d_j \quad (10)$$

where p_i (p_j) is the period of $\mathcal{P}_i$ ($\mathcal{P}_j$), and $g_{i,j}$ is the greatest common factor of the periods p_i and p_j .

On the basis of the above non-overlapping conditions, we can describe all the strictly periodic partitions' constraints as the following inequalities:

$$\begin{aligned} \forall i, j \in [1, n_1], i \neq j, i < j : \\ 0 \leq o_i \leq p_i - d_i, \\ 0 \leq o_j \leq p_j - d_j, \\ d_i \leq (o_j - o_i) \bmod g_{i,j} \leq g_{i,j} - d_j. \end{aligned} \quad (11)$$

To simplify the inequality constraints, the Eq. (10) is linearized as

$$d_i \leq (o_j - o_i) - g_{i,j} \times e_{i,j} \leq g_{i,j} - d_j \quad (12)$$

where $e_{i,j} = \lfloor \frac{o_j - o_i}{g_{i,j}} \rfloor$ and we have

$$\left\lfloor \frac{d_i - p_i}{g_{i,j}} \right\rfloor \leq e_{i,j} \leq \left\lfloor \frac{p_j - d_j}{g_{i,j}} \right\rfloor \quad (13)$$

Hence the inequality Eq. (11) can be described as the simpler linear inequalities:

$$\begin{aligned} \forall i, j \in [1, n_1], i \neq j, i < j : \\ 0 \leq o_i \leq p_i - d_i, \\ 0 \leq o_j \leq p_j - d_j, \\ d_i \leq (o_j - o_i) - g_{i,j} \times e_{i,j} \leq g_{i,j} - d_j \\ \left\lfloor \frac{d_i - p_i}{g_{i,j}} \right\rfloor \leq e_{i,j} \leq \left\lfloor \frac{p_j - d_j}{g_{i,j}} \right\rfloor. \end{aligned} \quad (14)$$

A valid partition schedule $\mathcal{S}$ should satisfy the linear constraints in Eq. (14). We adopt the following objective function

$$f(O) = \sum_{i=1}^{n_1} o_i \quad (15)$$

where the unknown variable vector $O = \langle o_1, o_2, \dots, o_{n_1} \rangle$ contains all the offsets of partition windows in the partition schedule. The objective function $f(O)$ is the sum of the offsets of all strictly periodic partitions, thus preferring partition time windows at lower offsets in the scheduling table $\mathcal{S}$.

Finally, given the unknown variable vector $O = \langle o_1, o_2, \dots, o_{n_1} \rangle$, the linear programming model is formulated as:

$$\begin{aligned} & \text{Minimize } f(O) = \sum_{i=1}^{n_1} o_i \\ & \text{subject to} \\ & \forall i, j \in [1, n_1], i \neq j, i < j : \\ & 0 \leq o_i \leq p_i - d_i, \\ & 0 \leq o_j \leq p_j - d_j, \\ & d_i \leq (o_j - o_i) - g_{i,j} \times e_{i,j} \leq g_{i,j} - d_j \\ & \left\lfloor \frac{d_i - p_i}{g_{i,j}} \right\rfloor \leq e_{i,j} \leq \left\lfloor \frac{p_j - d_j}{g_{i,j}} \right\rfloor. \end{aligned} \quad (16)$$

where o_i and o_j are the offsets of any strictly periodic partitions $\mathcal{P}_i$ and $\mathcal{P}_j$ respectively, d_i and d_j are the duration of $\mathcal{P}_i$ and $\mathcal{P}_j$ respectively, p_i and p_j denote the periods of $\mathcal{P}_i$ and $\mathcal{P}_j$ respectively, $g_{i,j}$ is the least common multiple of the periods p_i and p_j , and $e_{i,j} = \left\lfloor \frac{o_i - o_j}{g_{i,j}} \right\rfloor$ is a constructed unknown variable for linearizing the schedulability constraints.

Scheduling loosely periodic partitions

Assume that there are $n_2 = n - n_1$ loosely periodic partitions in the system. Given the set of loosely periodic partitions $\Omega_2 = \{\mathcal{P}_i \mid i = n_1 + 1, n_1 + 2, \dots, n\}$, the n_2 partitions from $\mathcal{P}_{n_1+1}$ to $\mathcal{P}_n$ are arranged in ascending order of the upper bounds of their jitter.

For any scheduling table $\mathcal{S} = \{W_i \mid i = 1, 2, \dots, n\}$ (in “Related work”), we define a partition window list as $\mathcal{L} = \{\langle \rho_k, \langle o_k, d_k \rangle \rangle \mid k = 1, 2, \dots, \omega; \rho_k \in \{\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_n\}; o_k = o_1, o_2, \dots, o_\omega; d_k = d_1, d_2, \dots, d_\omega\}$ where ω is the number of partition time windows in $\mathcal{S}$, $\langle o_k, d_k \rangle$ is the k -th window in the list, and ρ_k is the partition of $\langle o_k, d_k \rangle$. In the list $\mathcal{L}$, partition time windows are sorted in ascending order of the offsets o_k , satisfying the inequality $o_1 < o_1 + d_1 \leq o_2 < o_2 + d_2 \leq \dots < o_\omega < o_\omega + d_\omega$.

After building the scheduling table $\mathcal{S}^{(st)} = \bigcup_{i=1}^{n_1} \mathcal{S}_i$ for strictly periodic partitions, the time windows of loosely periodic partitions are generated by performing the following steps (outlined in Fig. 6):

- (1) The schedulability variable *SCHED* is first set to *True*. The partition window list $\mathcal{L}$ is initialized as $\mathcal{L} = \mathcal{L}_1 \cup \{\langle _, \langle 0, 0 \rangle \rangle, \langle _, \langle M, 0 \rangle \rangle\}$, where $\langle 0, 0 \rangle$ and $\langle M, 0 \rangle$ are two auxiliary time windows with length 0, the placeholder “ $_$ ” indicates that the time window does not belong to any partition, M represents the major time frame, and $\mathcal{L}_1$ is the partition window list of $\mathcal{S}^{(st)}$. Note that $\mathcal{L}$ always keeps the ascending order of its time window offsets. Hence the order of elements from $\mathcal{L}_1$ is still preserved in $\mathcal{L}$. Two auxiliary windows $\langle 0, 0 \rangle$ and $\langle M, 0 \rangle$ is settled at the first and last position of $\mathcal{L}$ respectively.
- (2) For each partition $\mathcal{P}_i \in \Omega_2$, the steps (3) to (12) are performed to create all the partition time windows in W_i .
- (3) For each period in the major time frame M (M/p_i periods), perform the steps (4) to (11) so as to satisfy the periodic budget.
- (4) For the j th period ($j = 0, 1, \dots, \frac{M}{p_i} - 1$), we define a current offset variable $o = jp_i$ and a current budget variable $b = b_i$.
- (5) Repeat the steps (6) to (9) if the current budget satisfies $b > 0$ and the schedulability variable *SCHED* = *True*.
- (6) Search $\mathcal{L}$ for available time slots, looking for two adjacent partition time windows $\langle o_k, d_k \rangle$ and $\langle o_{k+1}, d_{k+1} \rangle$ such that $\langle \rho_k, \langle o_k, d_k \rangle \rangle, \langle \rho_{k+1}, \langle o_{k+1}, d_{k+1} \rangle \rangle \in \mathcal{L}$ and $o_k \leq o < o_{k+1}$.
- (7) If such windows are not found in (6), we will update the schedulability variable *SCHED* = *False* and go to step (5), otherwise we should proceed to perform the following steps (8) and (9).
- (8) Update the current offset $o = \max(o, o_k + d_k)$ and define the current available duration $a = o_{k+1} - o$. Add a new window $\langle o, \min(a, b) \rangle$ to the partition $\mathcal{P}_i$ by inserting the element $\langle \mathcal{P}_i, \langle o, \min(a, b) \rangle \rangle$ into $\mathcal{L}$.
- (9) Update the current budget $b = \max(b - a, 0)$. Update the current offset o : if $a > b$, then $o = o + b$; Otherwise, update $o = o_{k+1} + d_{k+1}$. Return to step (5).
- (10) If $o > (j + 1)p_i$, we update the schedulability variable *SCHED* = *False*.
- (11) Update j to the next period. If $j < \frac{M}{p_i} - 1$, we will return to step (3).
- (12) Otherwise, we will explore the next partition $\mathcal{P}_{i+1}$ and return to step (2).
- (13) Perform $\mathcal{L} = \mathcal{L} \setminus \{\langle _, \langle 0, 0 \rangle \rangle, \langle _, \langle M, 0 \rangle \rangle\}$ that removes the auxiliary time windows. Calculate the partition scheduling table $\mathcal{S}$ based on $\mathcal{L}$.
- (14) Return the partition scheduling table $\mathcal{S}$ and the schedulability variable *SCHED*.

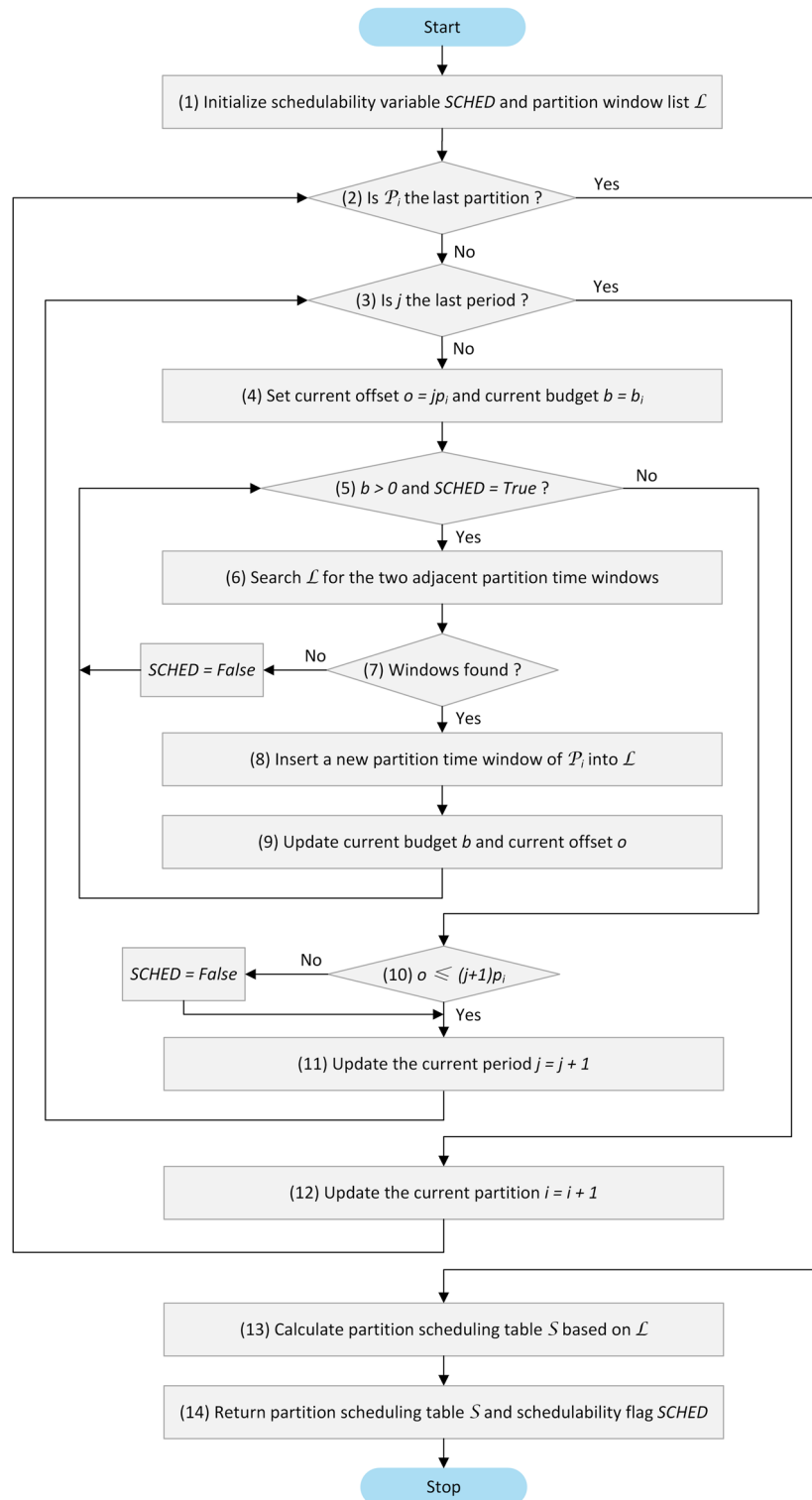


Figure 6. Flowchart of generating partition scheduling table for loosely periodic partitions.

Finally, the schedulability variable *SCHED* indicates whether the system is schedulable in *S*. The partition scheduling table *S* is successfully generated if *SCHED* = *True*, providing precisely periodic time slots for the strictly periodic partitions in $\Omega_1 = \{\mathcal{P}_i \mid i = 1, 2, \dots, n_1\}$ and only periodic time supply for the loosely periodic partitions in $\Omega_2 = \{\mathcal{P}_i \mid i = n_1 + 1, n_1 + 2, \dots, n\}$. For loosely periodic partitions, the generation algorithm provides a “best-effort” scheduling service. Therefore, the time windows of a loosely periodic partition may not strictly meet its timing demand, and the final schedulability variable, *SCHED*, is returned as *False* to alert the user in this case.

Formal model of IMA scheduling systems

The partition scheduling system of IMA is modeled as a set of timed automata in UPPAAL. This section first outlines the layered structure of the formal models and then presents the major templates of the model.

Layered structure of the IMA scheduling model

The compositionality in model checking requires the optimizer to handle each partition's verification independently, combining the local results to derive a global schedulability conclusion. Thus the formal model of the IMA scheduling system should support the independent description and verification for each partition.

To apply compositional model checking, we employ timed automaton models to delineate the task execution behaviors within each partition. As illustrated in Fig. 7, there is a set of timed automaton templates $\mathcal{M} = \{\mathcal{P}, \mathcal{R}, \mathcal{T}\}$ that consists of a partition scheduler $\mathcal{P}$, task scheduler $\mathcal{R}$ and task template $\mathcal{T}$. The templates in $\mathcal{M}$ are created to model the generic behavior of a partition. A scheduling table $\mathcal{S}$ from the above parallel genetic algorithm is divided into n sets of partition time windows W_i for corresponding partition $\mathcal{P}_i$. Each W_i of the scheduling table $\mathcal{S}$ can be independently imported into the template set $\mathcal{M}$, establishing the instance model of the partition $\mathcal{P}_i$. Finally, n instance models $\mathcal{M}_1, \dots, \mathcal{M}_n$ are constructed simultaneously. Each of them representing a distinct partition undergoes an independent model checking and provides a compositional schedulability conclusion.

The template $\mathcal{P}$ describes scheduling actions for a partition, $\mathcal{R}$ delineates scheduling behavior for the internal tasks within a partition, and $\mathcal{T}$ characterizes the behavior of a real-time task within the partition. The three types of models interact with each other via a few scheduling command/status channels, ensuring the cohesion of each component in the model. By importing the scheduling configuration into the corresponding templates, the instance model $\mathcal{M}_i$ of a partition $\mathcal{P}_i$ is created as follows:

- (1) The partition scheduler $\mathcal{P}$ is instantiated with the partition time window set W_i of $\mathcal{P}_i$ from the scheduling table $\mathcal{S}$, generating the partition scheduler model $\mathcal{Q}_i$.
- (2) The task scheduling policy R_i within partition $\mathcal{P}_i$ is embedded into the task scheduler $\mathcal{R}$, building the task scheduler model $\mathcal{R}_i$. The configuration of R_i determines the order of the ready queues and preemptive options.
- (3) The behavioral configuration T_j^i of task τ_j^i within partition $\mathcal{P}_i$ is integrated into the task template $\mathcal{T}$, producing the task model $\mathcal{T}_j^i$. The configuration T_j^i is implemented as a data structure that covers the main features of τ_j^i such as the task period, minimum and maximum execution time, deadline, priority, and core affinity.

By combining all the instance models $\mathcal{Q}_i$, $\mathcal{R}_i$ and $\mathcal{T}_j^i$ within the partition $\mathcal{P}_i$, we acquire the partition model $\mathcal{M}_i = \mathcal{Q}_i || \mathcal{R}_i || \bigcup_{j=1}^{m_i} \mathcal{T}_j^i$ that describes the behavior of the partition. The templates of timed automata are implemented in UPPAAL. We now discuss the templates of the partition scheduler, task scheduler and task template in detail.

Partition scheduler

The model of the partition scheduler template provides Time Division Multiplexing (TDM) scheduling services for a partition $\mathcal{P}_i$. The model activates this partition only within the partition time windows of S_i every major time frame.

The partition scheduling table $\mathcal{S}$ is recorded in an array of structures Sct , which specifies the time window(s) for each partition on the processor, ensuring that no partition exceeds its allotted time. Each element of the array represents a partition time window including three integer fields: *part*, *off*, and *dura*, where *part* is the partition

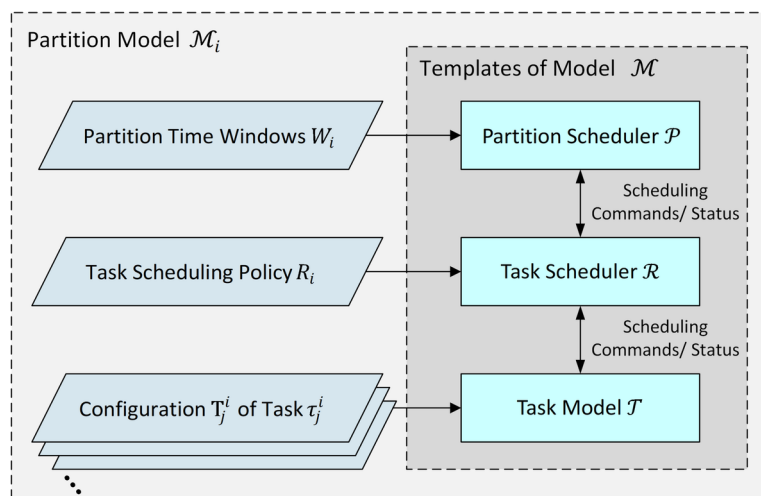


Figure 7. Modeling framework of IMA partition scheduling.

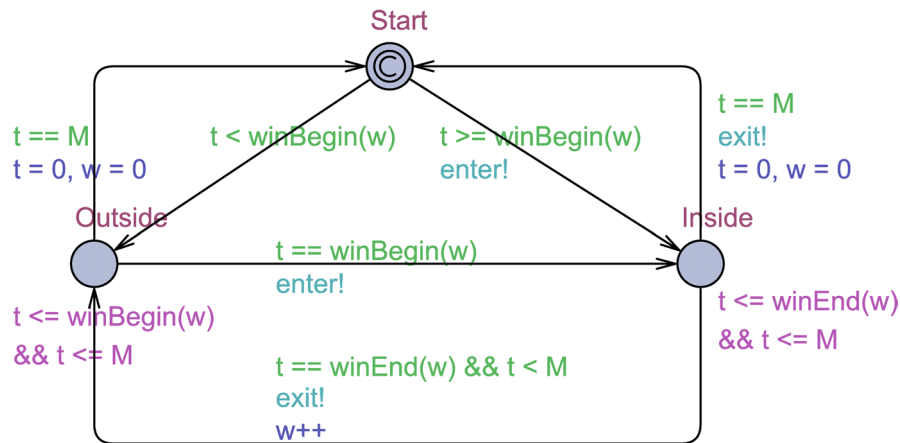


Figure 8. Template of partition scheduler.

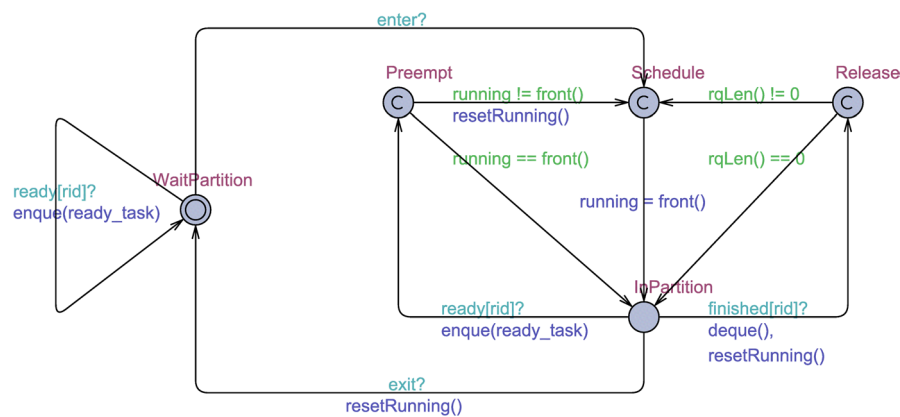


Figure 9. Template of task scheduler.

identifier, *offset* is the window's offset, and *dura* denotes its duration. The model defines two functions with a parameter *w*: *winBegin* and *winEnd* that return the start and end time of the *w*th window respectively according to the partition schedule table *Sct*. The major time frame is defined as an integer constant *M*. Since the partition scheduler is a time-driven model, a clock variable *t* is created to measure time within each *M*, thereby controlling partition scheduling actions.

As depicted in Fig. 8, the template of partition scheduler is comprised of three locations: *Start*, *Outside* and *Inside*. The initial location *Start* constructs a conditional control structure to determine the partition state at the start of each major time frame: if there is a time window of the partition that starts at the beginning of a major time frame, the model transitions to the *Inside* location; otherwise, it moves to the *Outside* location. The *Inside* and *Outside* locations represent the states in the partition and out of it, respectively. Subsequently, the model decides whether the current state is at *Inside* or *Outside* location in accordance with the value of clock *t*. On entering a new partition time window, the model immediately moves to *Inside* and notifies the task schedulers of the partition via a broadcast channel *enter*. By contrast, after leaving the current time window, the model transitions to *Outside* and employs a broadcast channel *exit* to notify the task schedulers, which block all the tasks executing within the partition.

Task scheduler

In an IMA system, each processor core activates the partition $\mathcal{P}_i$ synchronously and meanwhile enables the task scheduler of $\mathcal{P}_i$. As illustrated in Fig. 9, the task scheduler template describes how to schedule the tasks of $\mathcal{P}_i$, based on preemptive Fixed-Priority (FP) policy. The task scheduler model acquires the current state of the partition $\mathcal{P}_i$ from its partition scheduler model through two channels *enter* and *exit*. In addition, two channels *ready* and *finish* are used as scheduling commands to control the tasks of $\mathcal{P}_i$.

The task scheduler template declares an integer variable *running* to keep the task running on the processor core and defines a ready queue *rq* that stores all ready tasks in descending order of priority. *rq* is associated with four functions: *enqueue* inserts a new task identifiers into *rq* in the priority order; *dequeue* deletes the first element from *rq*; *front* and *rqLen* return the first task and the length of *rq*, respectively.

There are two major locations, *WaitPartition* and *InPartition*, in the template. *WaitPartition* is the initial location. The model at *WaitPartition* is out of the partition and will not react to any new ready commands to schedule tasks to run. In contrast, the state of *InPartition* means the current time is within the partition, tasks can be scheduled onto the processor.

A task is bound to the processor core *rid* according to the processor affinity property of the task. A newly ready task transfers a *ready* command to its task scheduler model. The task scheduler always assigns the processor to the highest-priority task in *rq*. When a new task with higher priority than the first element in *rq* becomes ready, the task scheduler places the task into *rq*, halts the currently running task via the *resetRunning* function, and assigns the newly selected task for execution by updating the *running* variable. When a task finishes its execution on the processor, it also conveys a *finish* command to its task scheduler. The first element in *rq* will be scheduled if the ready queue is not empty.

Task template

The task template in Fig. 10 describes the temporal behavior of a real-time task τ_j^i . This template defines two clock variables *t* and *x*. The clock *t* measures the time within each task period and detects whether the response time satisfies the deadline. The clock *x* calculates the execution time of the task occupying the processor core within the current period. The clock *x* is active only while the task is running. It can be suspended by setting its derivative value x' to zero when the task is not executing on the processor core.

There are three basic locations *WaitInitialOffset*, *Ready* and *WaitNextPeriod* in the task template. It starts at *WaitInitialOffset* and stays for the initial offset I_j^i of the task τ_j^i . Subsequently, the task becomes ready, making the model move to the *Ready* location and emitting a *ready* notification. The model stays at *Ready* for an undetermined time between the best-case execution time E_j^i and the worst G_j^i , which is returned by the function *BCET* and *WCET* respectively. When the task finishes its execution in this period, its model sends a message to the task scheduler via the *finish* channel, meanwhile transitioning to *WaitNextPeriod*. The model should return to the initial location *WaitInitialOffset* and repeat the above actions in the next period.

We define a special location *Unschedulable* that means the task τ_j^i misses its deadline D_j^i . A Boolean variable *deadlineMissed* should be assigned true once any task model enters the *Unschedulable* location. Thus, we check the schedulability of the system by verifying the following temporal property in UPPAAL:

$$A[] \text{ not } \text{deadlineMissed} \quad (17)$$

which denotes that any of the task models will never reach the *deadlineMissed* states in all paths.

Although the current evaluation focuses on periodic tasks, the proposed task modeling approach can be extended to handle non-periodic (e.g., sporadic or aperiodic) tasks as well. In all cases, deadline guarantees are enforced by modeling relative deadline constraints using timed automata: each task includes a clock that tracks its execution time, and a violation leads to a transition into an “Unschedulable” state, which is excluded by the model checker during verification.

Experiments

In this section, we apply the model-based method to the partition scheduling of two IMA systems. An experimental platform is built on a cluster computing environment. We first attempt to optimize the partition scheduling of a simple avionics system, which shows the capability of our model-based method to find the globally optimal solution. Subsequently, a more complex avionics workload is scheduled by the model-based method to acquire a feasible scheduling table within acceptable time and memory constraints. The traditional analytical methods and classical genetic algorithm are also compared with our methods in the experiments.

Experimental platform

The optimizer is implemented in Python and deployed in a cluster computing environment, whose highly parallelization speeds up the model-based analysis as well as the search for the optimal solutions.

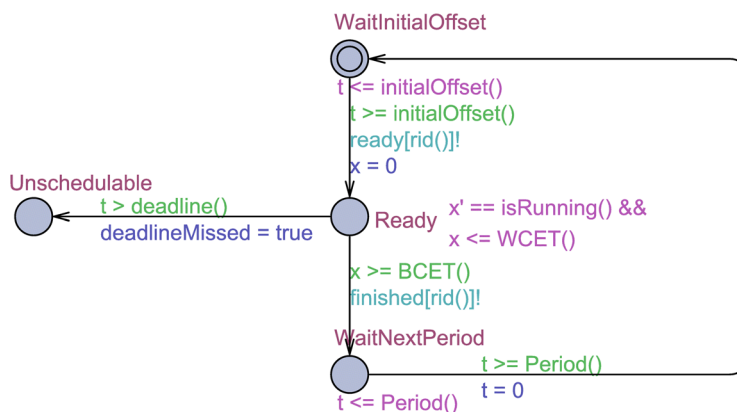


Figure 10. Task template.

Configuration	Hardware/software information
Nodes	4 workstations
Processors	AMD Ryzen Threadripper 7970X 4.0GHz
Number of cores	32 cores / processor
Memory	512GB / node
Disk size	16TB
Interaction	40Gbps QDR InfiniBand
OS	Ubuntu Server 22.04.4
UPPAAL	64-bit Linux version 4.1.25
Python	Python 3.12.3
Libraries	OpenMPI 4.1.6, mpi4py 3.1.6

Table 1. The configuration of the cluster in experiments.

Partition	Task set					
	Task	Period	WCET	Deadline	Priority	Core
P_1	τ_1^1	1600	80	1000	4	0
	τ_2^1	2400	120	2000	3	0
	τ_3^1	3200	160	3000	2	0
	τ_4^1	4800	240	4000	1	0
P_2	τ_1^2	1600	80	1000	4	1
	τ_2^2	2400	120	2000	3	1
	τ_3^2	3200	160	3000	2	1
	τ_4^2	4800	240	4000	1	1

Table 2. Task sets in experiment 1.

Table 1 shows the configuration of the cluster computing environment. In the cluster, there are four nodes that interact with each other via 40Gbps InfiniBand connections. To support the parallel model checking and genetic evolution, the cluster is equipped with $32 \times 4 = 128$ processor cores. Each node is a workstation allocated 512GB of dedicated memory and shares 16TB of disk space.

The software configuration is built on the Ubuntu Linux operating system, utilizing the Message Passing Interface (MPI) protocol to partition and aggregate data across computing nodes in the cluster. The MPI protocol services are implemented using the mpi4py and Open MPI library. The model checker UPPAAL of the optimizer is the 64-bit Linux version.

Experiment on a simple avionics system

The first experiment is to perform the optimal scheduling for the avionics task sets in⁵¹. As shown in Table 2, there are two partitions in this simple avionics system, where the time unit is 1ms. We generate the partition scheduling table by using the following four methods.

- (1) Exhaustive search (ES): In⁵¹, the schedulability of the system was determined using response time analysis. The method involved testing all possible integer combinations of partition periods within the range [40,2000]. For each combination, a binary search strategy was employed to find the minimum partition budget from the highest to the lowest priority partition. Although exhaustive search can find the global optimal solution, it is only practical for simple systems. Its effectiveness decreases significantly as the parameter space and system complexity increase.
- (2) Geometric programming: In¹⁰, Geometric Programming (GP) was used to model and solve the optimization problem of partitioned scheduling systems. This method is suitable for addressing nonlinear and non-convex optimization problems but is limited to handling simple sets of periodic tasks with fixed execution times. For this study, a GP optimization program for the avionics system was implemented using the Python library Gpkit 1.1.
- (3) Basic genetic algorithm: Two genetic algorithms were utilized as follows. The Basic Genetic Algorithm⁵² and the Breeder Genetic Algorithm (BGA)⁵³, referred to as “GA1” and “GA2,” respectively. Both algorithms used binary encoding for individuals. In GA1, the crossover probability for each bit was set at 0.5 and the mutation probability at 0.2. In GA2, the proportion of truncation selection $T\%$ was set to 50%, the weight constraint for the intermediate recombination operator was 0.5, the standard deviation for the Gaussian mutation operator was 10, and the mutation probability was 0.2.

Method	Demand vector	Processor occupancy	Optimal
ES	(1600,341,1600,341)	45.1%	✓
GP	(483, 137, 488, 152)	67.8%	
GA 1	(630, 170, 1260, 350)	59.5%	
GA 2	(1200, 280, 800, 160)	47.5%	
MBGA	(1600,341,1600,341)	45.1%	✓

Table 3. Scheduling results in experiment 1.

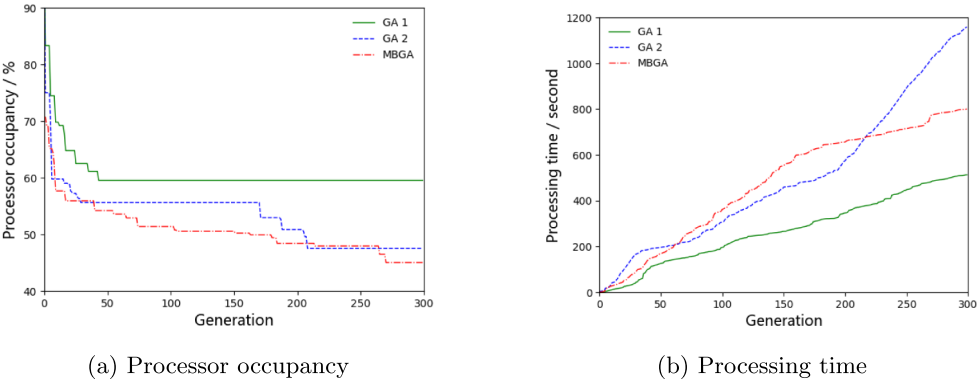


Figure 11. Evolution of the processor occupancy and processing time in experiment 1.

- (4) Model-based method in this paper (MBGA in short): When scheduling with the proposed method, individuals are also binary encoded. The base of the exponential ranking selection is set to $c = 0.8$, and the constraint factor for recombination is $d = 0.5$. The standard deviation of the mutation is defined as $\sigma_{2i-1} = \pi/32, \sigma_{2i} = 2000$.

Table 3 summarizes the results of Experiment 1. The MBGA method found the same global optimal solution as the exhaustive search with a minimal processor occupancy 45.1%. However, the two traditional genetic algorithms only identified two local optimal solutions, associated with higher processor occupancy 59.5% and 47.5%, respectively. When compared to the optimization approach based on formal models, the geometric programming method—using a less precise analytical model of resource supply and demand—produced a schedulable solution, but with a significantly higher occupancy of 67.8%, wasting the processor time severely.

Figure 11 depicts the changes of the processor occupancy and cumulative processing time in Experiment 1. Clearly, GA1 prematurely converged to a local optimum with the processor occupancy 59.5% after approximately 50 generations. By contrast, GA2 was also stuck in a local optimum for a long period of 152 generations. However, at the 167th generation, a random mutation enabled it to discover an individual with lower processor utilization. Following this, GA2 conducted local searches based on this individual, making slight progress, but soon reached a stable state again and ultimately converged to a local optimum with the processor occupancy 47.5%. Conversely, MBGA consistently identified better solutions throughout the optimization process, eventually converging to the global optimum.

Experiment on a concrete avionics system

Experiment 2 tackles the partition scheduling problem for the complex IMA task set outlined in Table 4⁵⁴. This ARINC 653 partition scheduling might seem like a simple task scheduling problem, but it is actually a complex combinatorial optimization challenge. For a single hosted application and its corresponding partition, the optimal scheduling parameters, such as the partition period and execution budget, can be determined using exact model-based schedulability verification. However, finding a globally optimal set of partition scheduling parameters is often intractable because the combination of local optimization of individual partitions does not guarantee a globally optimal solution¹⁰. Consequently, the scheduling parameters for each partition cannot be selected independently; instead they must be jointly optimized. A brute-force search of the solution space would only be practical when certain parameters are fixed and/or the number of applications is small. In concrete IMA projects, the solution space is typically too large for such a search to be feasible. Thus, this experiment compares the optimization of the MBGA method with the empirical scheduling scheme detailed in⁵⁴. The context-switching overhead between system partitions is set at 0.2ms.

The detailed experimental configuration is as follows:

- (1) Empirical scheduling

Partition	Task set							
	Task	Period	Initial offset	BCET	WCET	Deadline	Priority	Core
P_1	τ_1^1	25	2	0.9	1.5	25	5	0
	τ_2^1	50	3	0.2	0.4	50	4	0
	τ_3^1	50	3	2.7	4.2	50	3	0
	τ_4^1	50	0	0.1	0.2	50	2	0
	τ_5^1	120	0	0.7	1.1	120	1	0
P_2	τ_1^2	50	0	1.9	3.0	50	5	1
	τ_2^2	50	2	0.7	1.1	50	4	1
	τ_2^3	100	0	0.1	0.2	100	3	1
	τ_2^4	100	10	1.0	1.6	100	2	1
P_3	τ_1^3	25	0	0.5	0.8	25	5	2
	τ_2^3	50	0	0.7	1.1	50	4	2
	τ_3^3	50	0	1.0	1.6	50	3	2
	τ_4^3	100	11	0.8	1.3	100	2	2
P_4	τ_1^4	25	3	0.7	1.2	25	5	3
	τ_2^4	50	5	1.2	1.9	50	4	3
	τ_3^4	50	25	0.1	0.2	50	3	3
	τ_4^4	100	11	0.7	1.1	100	2	3
	τ_4^5	200	13	3.7	5.8	200	1	3
P_5	τ_1^5	50	0	0.7	1.1	50	6	0
	τ_2^5	50	2	1.2	1.9	50	5	0
	τ_3^5	200	0	0.6	0.9	200	4	0
	τ_4^5	200	14	1.5	2.4	200	3	0

Table 4. Task set in experiment 2.

Method	Demand vector	Processor occupancy
Empirical	(25, 4.8, 25, 4.8, 25, 4.8, 25, 4.8, 25, 4.8)	100%
MBGA	(25, 4.9, 25, 4.7, 25, 3.4, 25, 4.5, 50, 4.5)	83%

Table 5. Scheduling results in experiment 2.

First, all partition periods are defined as 25 ms, which is the shortest task period within the partitions and the greatest common divisor of most task periods. Each partition is allocated a 5ms time window within every partition period. A partition schedule is created based on this, using the shared partition period as the major time frame 25 ms. Within each major time frame, the five partition time windows are arranged sequentially according to priority. A context-switching overhead of 0.2ms is inserted at the beginning of each partition time window, reducing each window's length to 4.8ms.

(2) MBGA method

Considering the large parameter space and the extended processing time per generation, the MBGA method is configured with a new population size 256 and a maximum iteration count 200 generations. The operator configurations from Experiment 2 are retained, with the base of the exponential ranking selection operator $c=0.8$, the constraint factor of the recombination operator $d=0.5$ and standard deviation of the mutation $\sigma_{2i-1} = \pi/32, \sigma_{2i} = 100$.

Table 5 demonstrates the results of Experiment 2. Owing to the large unknown parameter space and high processor load, the concrete avionics system presented greater challenges in identifying solutions that met the system's schedulability constraints compared to Experiment 1. The traditional empirical scheduling approach

consumed all processor time but still failed to generate a schedulable table. The counterexample produced by UPPAAL during schedulability verification revealed that task τ_3^1 missed its deadline in the instant of 50.2 ms.

By contrast, the MBGA method utilizes genetic algorithm to explore the solution space of the scheduling table, iteratively creating and invoking a substantial number of parallel UPPAAL verifier processes to improve processing efficiency. It was executed for a total of 200 generations, lasting 26 hours and 23 minutes, finally selecting the best individual with a lower processor occupancy 83%.

Comparison experiment with parameter synthesis

IMITATOR²⁹ is a powerful tool to perform automated parameter synthesis for concurrent timed systems in PTA. The tool computes constraint sets over parameters that ensure the system satisfies some properties. It is a straightforward way to apply automated parameter synthesis of IMITATOR to gain the feasible parameter regions that guarantee the schedulability property of systems. To do so, we extended the experimental comparison and tried to apply the state-of-the-art parametric model checker IMITATOR 3.3 to the same set of benchmark tasks used in our framework. We built a functionally equivalent model in IMITATOR PTA and ran it under the same experimental conditions.

However, the parameter synthesis using IMITATOR is shown computationally infeasible due to the tool's reliance on rational-valued parameters. In our tests, although IMITATOR can quickly verify the schedulability of models with fixed parameters, it is computationally infeasible when used for parameter synthesis. In fact, IMITATOR cannot terminate even when the parameter domain is restricted to a very narrow range. Using the simple system model from Experiment 1, we constrained the partition budget to a small interval of length 1 (e.g., [1000,1001]). Despite this, IMITATOR still failed to terminate after more than three weeks (over 504 hours) of continuous execution.

This result highlights the computational challenges of applying automated parameter synthesis methods to complex high-dimensional scheduling problems and reinforces the efficiency advantage of our approach, which uses fixed-valued parameters to complete an efficient exploration, while IMITATOR performs symbolic analysis on infinite rational parameter space.

Discussion

The comparative experiment indicates that formal models, which describe system behavior directly, offer greater expressiveness and accuracy compared to analytical models. Consequently, the optimization results obtained through formal methods (GA, ES, and MBGA) are generally superior to those derived from analytical models (GP).

Among the two optimization methods based on analytical models, GP employs a relatively conservative approximation of the resource demand-supply relationship to satisfy the mathematical requirements of geometric programming, resulting in more pessimistic optimization outcomes.

Within the three optimization methods based on formal models, GA benefits from the precise description of system behavior provided by formal models, enabling the discovery of several locally optimal solutions with low processor utilization. However, due to the low search efficiency of its operators, considerable processor time is still wasted. In contrast, MBGA employs specialized evolutionary operators and an adaptive mechanism for operator configuration, allowing more accurate exploration of the system's schedulability boundaries. In the comparative experiment, MBGA consistently found the same global optimal solutions as ES.

To ensure that each generated schedule remains feasible with respect to ARINC 653 constraints, the proposed framework integrates UPPAAL model checking into the MBGA evaluation loop. Each candidate chromosome, representing a scheduling table, is encoded into a timed automata model and verified for schedulability before being accepted for fitness calculation. This built-in verification step guarantees that the optimization process respects all timing and structural constraints defined by the IMA architecture.

Additionally, our experimental results indicate that MBGA maintains structural stability across multiple runs. Despite random initialization, the final schedules produced for the same task set showed consistent processor occupancy rates and similar time window distributions, indicating robustness in the search behavior. This characteristic is particularly important in safety-critical systems, where deterministic or near-deterministic scheduling structures are often preferred.

The performance of MBGA was also evaluated under scenarios of increased system complexity. Taking the complex avionics system optimized in Experiment 2 as an example, with a large parameter space (10 dimensions), higher task load (57.07%), and complex communication and interaction relationships among tasks, exhaustive search is no longer feasible for such a vast parameter space. Nevertheless, the MBGA proposed in this paper is still able to find partition scheduling tables with lower processor utilization, demonstrating excellent optimization performance and applicability.

To further compare MBGA with state-of-the-art tools, we constructed a functionally equivalent IMITATOR model and applied it to the same benchmark task set. However, due to the lack of parallelization support and the computational complexity of symbolic region exploration, the model failed to terminate even after several weeks of execution on a high-performance cluster. This observation highlights the current limitations of parametric model checkers like IMITATOR in solving large-scale scheduling optimization problems, and reinforces the practicality of our UPPAAL-based GA-accelerated approach for scalable partition scheduling.

To improve computational efficiency, the proposed method already incorporates several domain-specific strategies. The recombination and mutation operators are designed based on the proportional relationship between partition parameters, which enhances convergence efficiency over traditional genetic algorithms. In addition, the initial parameter search space is constrained using design patterns derived from the ARINC 653 standard, such as common ratios between partition and task periods.

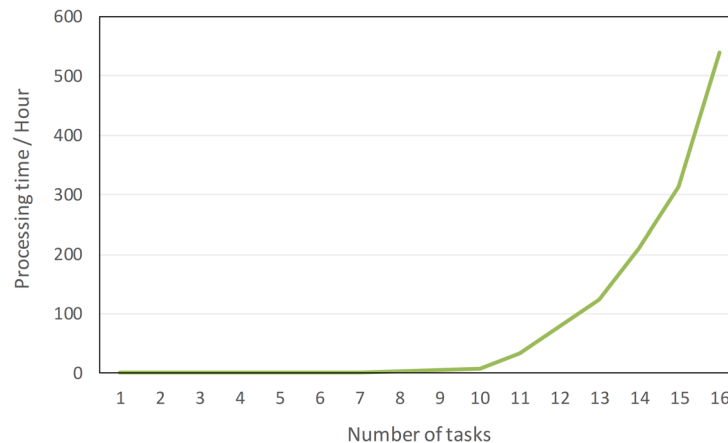


Figure 12. Computation time versus number of tasks, revealing the scalability limitations of model-based analysis for high-density partitions.

Moreover, in avionics engineering, scheduling tables are typically synthesized during the early design phase and remain relatively fixed throughout the aircraft's operational lifecycle to ensure system determinism and certification compliance. Considering that aircraft development often spans several years, a design-time computation cost of several days is acceptable when it enables formally verified, globally optimized scheduling solutions.

Computational complexity analysis

As shown in Algorithm 1, the computational complexity of the proposed algorithm MBGA is primarily associated with the four following components: the population size K , the number of generations G , the number of partitions N in the system, and the average time required for verifying the schedulability of each partition using UPPAAL, denoted as T . Since all the partitions of each individual in a population are processed parallel, the total computational complexity of the algorithm can be approximated as $O(G \cdot T)$.

In each generation, MBGA evaluates the entire population to guide the search toward feasible and optimal scheduling tables. The most time-consuming part is the schedulability verification process for each candidate chromosome. This process requires transforming the chromosome into a timed automata model and invoking UPPAAL to check whether all tasks meet their deadline constraints under the ARINC 653 standard.

Although model checking introduces a non-negligible cost compared to traditional optimization methods, the use of compositional modeling in UPPAAL helps mitigate the state-space explosion problem, particularly for systems with moderate task counts per partition. Moreover, MBGA avoids exhaustive enumeration of possible parameters by using stochastic search operators to efficiently explore the scheduling space.

Scalability evaluation

The scalability of the proposed model-based optimization method is primarily achieved through the use of parallelization at two levels.

During the optimization process, fitness evaluations—responsible for assessing each candidate partition scheduling table—are distributed across parallel computing resources. Since each evaluation is independent, this design allows efficient utilization of available cores or nodes, preventing processing time from increasing significantly even with larger population sizes.

Our compositional verification framework further improves scalability by enabling partition-level schedulability checks to be conducted in parallel. This strategy effectively mitigates the computational burden associated with increasing the number of partitions in the system.

However, the scalability of symbolic model checking within each partition remains a limiting factor. As the number of concurrent tasks in a partition increases, the complexity of the underlying timed automata model grows exponentially.

To examine this limitation, we conducted a scalability experiment that handled a single partition including the first k tasks from the online task dataset rand0000.stg of STG site⁵⁵, incrementally adding tasks from 1 up to all 50 tasks. As shown in Fig. 12, computation time increases steeply with the number of tasks, indicating the exponentially growing computational demands of symbolic model checking. When the task count exceeds 16, the processing time on our workstation surpasses 531 hours (over three weeks), rendering the approach impractical for high-density partitions. This result demonstrates a significant scalability bottleneck in model-based analysis.

Conclusions

This paper proposes a model-based method for partition scheduling in IMA systems using timed automata in UPPAAL. We conclude that this method provides precise schedulability bounds and feasible partition scheduling tables with lower processor occupancy compared to traditional analytical methods and classic

random search algorithms. The method also applies compositional verification for both strictly and loosely periodic partitions and improves the parallel genetic algorithm to efficiently explore the scheduling parameter space, thereby mitigating state space explosion in model checking. The experiments show that this method enhances the performance and efficiency of partition scheduling in IMA systems. The future studies include extending the framework to cover more platform features like shared resource contention and integrate mixed-criticality concepts within the ARINC 653-compliant systems while maintaining scalability.

Data availability

All data generated or analyzed during this study are included in this published article.

Received: 25 December 2024; Accepted: 19 August 2025

Published online: 26 August 2025

References

- Lukić, B., Ahlbrecht, A., Friedrich, S., Durak, U.: State-of-the-art technologies for integrated modular avionics and the way ahead. In *2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*, pp. 1–10 (2023). IEEE
- Wang, H. & Niu, W. A review on key technologies of the distributed integrated modular avionics system. *Int. J. Wireless Inf. Netw.* **25**, 358–369 (2018).
- Bieber, P., Boniol, F., Boyer, M., Noulard, E., & Pagetti, C. New challenges for future avionic architectures. *Aerospace Lab* **4**, 1 (2012).
- Committee, A. E. ARINC Specification 653P1-5 Avionics Application Software Standard Interface Part 1 - Required Services (2019).
- Kim, J.-E., Abdelzaher, T., & Sha, L. Schedulability bound for integrated modular avionics partitions. In *2015 Design, Automation and Test in Europe Conference and Exhibition (DATE)*, pp. 37–42 (2015). IEEE.
- Annighöfer, B. & Kleemann, E. Large-scale model-based avionics architecture optimization methods and case study. *IEEE Trans. Aerosp. Electron. Syst.* **55**(6), 3424–3441 (2019).
- Blikstad, M., Karlsson, E., Lööw, T. & Rönnerberg, E. An optimisation approach for pre-runtime scheduling of tasks and communication in an integrated modular avionic system. *Optim. Eng.* **19**, 977–1004 (2018).
- Craciunas, S. S. & Oliver, R. S. Combined task-and network-level scheduling for distributed time-triggered systems. *Real-Time Syst.* **52**, 161–200 (2016).
- Chen, J., Du, C., Xie, F. & Yang, Z. Schedulability analysis of non-preemptive strictly periodic tasks in multi-core real-time systems. *Real-Time Syst.* **52**, 239–271 (2016).
- Yoon, M.-K., Kim, J.-E., Bradford, R., & Sha, L. Holistic design parameter optimization of multiple periodic resources in hierarchical scheduling. In *2013 Design, Automation and Test in Europe Conference and Exhibition (DATE)*, pp. 1313–1318 (2013). IEEE.
- Alur, R. & Dill, D. L. A theory of timed automata. *Theoret. Comput. Sci.* **126**(2), 183–235 (1994).
- UPPAAL Home. (accessed on 15 June 2024). <http://www.uppaal.org/>.
- Boudjadar, J., Larsen, K. G., Kim, J. H., & Nyman, U. Compositional schedulability analysis of an avionics system using UPPAAL. In *International Conference on Advanced Aspects of Software Engineering*, pp. 140–147 (2014). CEUR Workshop Proceedings.
- Kim, J. H., Boudjadar, A., Nyman, U., Mikućionis, M., Larsen, K. G., Skou, A., Lee, I., & Thi Xuan Phan, L. Quantitative schedulability analysis of continuous probability tasks in a hierarchical context. In *Proceedings of the 18th International ACM SIGSOFT Symposium on Component-Based Software Engineering*, pp. 91–100 (2015).
- Boudjadar, A. et al. Statistical and exact schedulability analysis of hierarchical scheduling systems. *Sci. Comput. Program.* **127**, 103–130 (2016).
- Kim, J. H. et al. Optimizing the resource requirements of hierarchical scheduling systems. *Acm Sigbed Rev.* **13**(3), 41–48 (2016).
- Ahn, S. J., Hwang, D. Y., Kang, M. & Choi, J.-Y. Hierarchical system schedulability analysis framework using UPPAAL. *IEICE Trans. Inf. Syst.* **99**(8), 2172–2176 (2016).
- Han, P., Zhai, Z., Nielsen, B., & Nyman, U. A modeling framework for schedulability analysis of distributed avionics systems. arXiv preprint [arXiv:1803.11050](https://arxiv.org/abs/1803.11050) (2018).
- Singh, A., D'Souza, M., & Ebrahim, A. Conformance testing of ARINC 653 compliance for a safety critical rtos using UPPAAL model checker. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing*, pp. 1807–1814 (2021).
- Han, P., Zhai, Z., Nielsen, B. & Nyman, U. Model-based optimization of ARINC-653 partition scheduling. *Int. J. Softw. Tools Technol. Transfer* **23**(5), 721–740 (2021).
- Fersman, E., Krcal, P., Pettersson, P. & Yi, W. Task automata: Schedulability, decidability and undecidability. *Inf. Comput.* **205**(8), 1149–1172 (2007).
- David, A., Rasmussen, J. I., Larsen, K. G., & Skou, A. Model-based framework for schedulability analysis using uppaal 4.1. *Model-Based Design for Embedded Systems* (2018).
- Abdeddaïm, Y., Asarin, E. & Maler, O. Scheduling with timed automata. *Theoret. Comput. Sci.* **354**(2), 272–300 (2006).
- Abdeddaïm, Y., & Maler, O. Preemptive job-shop scheduling using stopwatch automata. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2002)*. Lecture Notes in Computer Science, vol. 2280, pp. 113–126 (2002).
- Le, T. T. H., Palopoli, L., Passerone, R. & Ramadian, Y. Timed-automata based schedulability analysis for distributed firm real-time systems: A case study. *Int. J. Softw. Tools Technol. Transfer* **15**, 211–228 (2013).
- Glonina, A. & Bahmurov, A. Stopwatch automata-based model for efficient schedulability analysis of modular computer systems. In *Parallel Computing Technologies* (ed. Malyshkin, V.) 289–300 (Springer, Cham, 2017).
- Sun, Y., Lipari, G., Soulat, R., Fribourg, L., & Markey, N. Component-based analysis of hierarchical scheduling using linear hybrid automata. In *2014 IEEE 20th International Conference on Embedded and Real-Time Computing Systems and Applications*, pp. 1–10 (2014). ISSN: 2325-1301.
- Cimatti, A., Palopoli, L., & Ramadian, Y. Symbolic computation of schedulability regions using parametric timed automata. In *Proceedings of the 2008 Real-Time Systems Symposium (RTSS 2008)*, pp. 80–91 (2008). IEEE.
- André, E. Imitator 3: Synthesis of timing parameters beyond decidability. In *Computer Aided Verification: 33rd International Conference, CAV 2021, Virtual Event, July 20 23, 2021, Proceedings, Part I*, pp. 552–565. Springer, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-030-81685-8_26.
- Sun, Y., Lipari, G., André, E., & Fribourg, L. Toward parametric timed interfaces for real-time components. In André, E., Frehse, G. (eds.) *Proceedings 1st International Workshop on Synthesis of Continuous Parameters, Grenoble, France*, 6th April 2014 (2014).
- Sun, Y., Soulat, R., Lipari, G., André, E. & Fribourg, L. Parametric schedulability analysis of fixed priority real-time distributed systems. In *Formal Techniques for Safety-Critical Systems* (eds Artho, C. & Ölveczky, P. C.) 212–228 (Springer, Cham, 2014).
- André, E. A unified formalism for monoprocessor schedulability analysis under uncertainty. In *Proceedings of the International Workshop on Formal Methods for Industrial Critical Systems and Automated Verification of Critical Systems (FMICS-AVoCS 2017)*, pp. 96–112 (2017).

33. André, E., Coquard, E., Fribourg, L., Jerray, J. & Lesens, D. Parametric schedulability analysis of a launcher flight control system under reactivity constraints. *Fund. Inform.* **182**(1), 1–24 (2021).
34. André, Chatain, T., Encrenaz, E., & Fribourg, L. An inverse method for parametric timed automata. In *Formal Modeling and Analysis of Timed Systems (FORMATS 2009). Lecture Notes in Computer Science*, vol. 5813, pp. 226–241. Springer, (2009). https://doi.org/10.1007/978-3-642-04368-0_17.
35. André, Lime, D., & Ramparison, M. Timed automata with parametric updates. In *Proceedings of the 18th International Conference on Application of Concurrency to System Design (ACSD)*, pp. 53–62. IEEE, (2018). <https://doi.org/10.1109/ACSD.2018.000-2>.
36. André, Bloemen, V., Petrucci, L., & Pol, J. Minimal-time synthesis for parametric timed automata. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2019), Part II. Lecture Notes in Computer Science*, vol. 11428, pp. 211–228. Springer, (2019). https://doi.org/10.1007/978-3-030-17465-1_12.
37. Ramparison, M. Parametric verification to ensure safe behaviour of connected devices. In *Presented at the 5th International Workshop on the Advancements in Model Driven Engineering & Software Engineering (AMDE 2024)*, Leuven, Belgium, October 28–30, 2024 *Procedia Computer Science*, vol. 251, pp. 502–507. Elsevier, (2024). <https://doi.org/10.1016/j.procs.2024.11.139>.
38. Han, P., Zhai, Z. & Zhang, L. A Model-Based Approach to Optimizing Partition Scheduling of Integrated Modular Avionics Systems. *Electronics* **9**(8), 1281. <https://doi.org/10.3390/electronics9081281> (2020).
39. Han, P., Hu, W., Zhai, Z., & Huang, M. A model-based optimization method of arinc 653 multicore partition scheduling. *Aerospace* **11**, 11. <https://doi.org/10.3390/aerospace11110915> (2024).
40. Cheptsov, V., & Khoroshilov, A. Robust resource partitioning approach for arinc 653 rtos. Technical report, arXiv preprint [arXiv:2312.01436](https://arxiv.org/abs/2312.01436). <https://arxiv.org/abs/2312.01436> (2023).
41. Zuepke, A., Krause, A. R., Mller, W., Nowak, N., & Pellerin, S. Autobest: A united autosar-os and arinc 653 kernel. In: *ERTS Embedded Real-Time Software and Systems* (2015). RealTime-at-Work, INRIA. <https://www.quest-lfs.uni-hannover.de/en/research/publications/details/fis-details/publ/9cff7261-6675-4f58-aa4c-e0dcf2b6b7e3>.
42. Lukic, D., Friedrich, R. & Durak, U. A streamlined approach toward automated generation and verification of arinc 653 configurations. *J. Aerosp. Inf. Syst.* <https://doi.org/10.2514/1.I011439> (2024).
43. Zhang, Y., Chen, Z., Chen, Q., & Yi, W. Refinement-based specification and analysis of multi-core arinc 653 systems. *Proceedings of the ACM on Programming Languages* **7**(OOPSLA), 222. <https://doi.org/10.1145/3617183> (2023).
44. Behrmann, G., David, A., Larsen, K.G.: A tutorial on UPPAAL. In: *Formal Methods for the Design of Real-time Systems*, pp. 200–236 (2004). Springer
45. Cassez, F., & Larsen, K. The impressive power of stopwatches. In *International Conference on Concurrency Theory*, pp. 138–152 (2000). Springer.
46. VanderLeest, S.H., & Matthews, D. C. Incremental assurance of multicore integrated modular avionics (ima). In *2021 IEEE/AIAA 40th Digital Avionics Systems Conference (DASC)*, pp. 1–9 (2021). IEEE.
47. Chen, J., Du, C. & Han, P. Scheduling independent partitions in integrated modular avionics systems. *PLoS ONE* **11**(12), 0168064 (2016).
48. Lipari, G., & Bini, E. A methodology for designing hierarchical scheduling systems. *J. Embedded Comput.* **1**(2), 257–269 (2005).
49. Beyer, H.-G. An alternative explanation for the manner in which genetic algorithms operate. *Biosystems* **41**(1), 1–15 (1997).
50. Korst, J., Aarts, E., Lenstra, J.K., & Wessels, J. Periodic Multiprocessor Scheduling. In: Aarts, E.H.L., Leeuwen, J., Rem, M. (eds.) *Parle 91 Parallel Architectures and Languages Europe. Lecture Notes in Computer Science*, pp. 166–178. Springer, Berlin, Heidelberg (1991). https://doi.org/10.1007/978-3-662-25209-3_12.
51. Davis, R., & Burns, A. An investigation into server parameter selection for hierarchical fixed priority pre-emptive systems. In *16th International Conference on Real-Time and Network Systems (RTNS 2008)* (2008).
52. Michalewicz, Z. *Evolutionary Computation 1: Basic Algorithms and Operators* (CRC Press, New York, 2018).
53. Mühlenbein, H., & Schlierkamp-Voosen, D. Predictive models for the breeder genetic algorithm i. continuous parameter optimization. *Evol. Comput.* **1**(1), 25–49 (1993).
54. Carnevali, L., Pinzuti, A. & Vicario, E. Compositional verification for hierarchical scheduling of real-time systems. *IEEE Trans. Softw. Eng.* **39**(5), 638–657 (2012).
55. Standard Task Graph Set. <https://www.kasahara.cs.waseda.ac.jp/schedule/>.

Acknowledgements

The authors would like to thank Brian Nielsen and Ulrik Nyman of Aalborg University for their great help on technical support for the model checker UPPAAL.

Author contributions

Conceptualization, J.C.; methodology, J.C.; software, P.H. and M.H.; validation, J.C.; formal analysis, J.C.; investigation, J.C.; resources, J.C.; data curation, J.C.; writing original draft preparation, J.C.; writing, review and editing, J.C.; visualization, J.C.; supervision, Z.Z.; project administration, P.H.; funding acquisition, P.H. and M.H. All authors have read and agreed to the published version of the manuscript.

Funding

This research was funded by the Industrial Science and Technology Research Project of Henan Province under grant number 222102210024 and the Doctoral Fund Project of Zhengzhou University of Light Industry under grant number 2021BSJJ028.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Z.Z. or M.H.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025